

Capstone Engineering Design Document and Test Plan

Smart Distributed Acoustic Calibration System (SDACS)

Kyle Dick (ID: 7206287), David Toma (ID: 8823349), Amy Wentzell (ID: 8820998)

Winter 2026 – Level 10

Bachelor of Applied Engineering – Electronic Systems Engineering, Conestoga
College

EECE74125 - Capstone Project I

Professor Dr. So-Ra Chung, Professor Dr. Reda Fayek

April 16, 2026

Table of Contents

1. INTRODUCTION	6
1.1. Purpose.....	6
1.2. Project Design Description	6
1.1.1. Server	7
1.1.2. Source Node.....	7
1.1.3. Recording Nodes.....	7
1.1.4. Client (Website / Application)	7
1.3. System Requirements.....	12
2. SYSTEM DESIGN	13
2.1. Functional Units.....	13
2.1.1. Recording Nodes.....	13
2.1.2. Server	15
2.1.3. Application (under development)	17
2.2. Test Plans[40][42].....	18
2.2.1. Recording Nodes (Summary of both Test Plans)	19
2.2.2. Server (Node-RED Debug).....	22
2.2.3. Application (under development)	24
2.2.4. Off-the-shelf Components: Microcontroller, Adafruit Metro S3	25
2.2.5. Off-the-shelf Components: PC, Raspberry Pi 5.....	26
2.3. Safety, Societal and Environmental Issues	27
2.3.1 Safety Considerations	27
2.3.2 Environmental Considerations.....	27
2.3.3 Societal Considerations.....	28
3. DESIGN OF FUNCTIONAL UNITS	29

3.1.	Software	29
3.1.1.	Server Design – Inputs and Outputs	29
3.1.2.	NodeRED Process Flow	30
3.1.3.	Server Design – Investigation and Decisions Made	37
3.1.4.	MATLAB Data Analysis (under development).....	38
3.2.	Hardware.....	39
3.2.1	System Overview	39
3.2.2	Hardware Architecture.....	40
3.2.3	Digital MEMS Microphone, ICS-43434.....	41
3.2.4	Temperature and Humidity Sensor, Sensirion SHT41	42
3.2.5	LiPo Battery, 3.7V with MAX17048 Fuel Gauge	42
3.2.6	MicroSD Card.....	43
3.2.7	Custom PCB Design	44
3.2.8	Schematics	45
3.2.9	PCB Layout.....	49
3.3.	Firmware.....	50
3.3.1	Firmware Design.....	50
3.3.2	Inputs and Outputs	51
3.3.3	Firmware Process Flow.....	52
3.3.4	Firmware System Architecture	57
3.3.5	Firmware Design Decisions and Justifications	59
3.3.6	Key Features Implemented	62
3.3.7	Limitations and Future Improvements.....	62
3.4.	Enclosure Design	63
3.4.1.	Battery Port	64

3.4.2.	MCU Housing.....	65
3.4.3.	Lid.....	66
4.	RISK ANALYSIS AND RISK MANAGEMENT PLAN.....	67
4.1.	Technical, Quality, and Performance Risks.....	67
4.1.1.	Competitive Landscape.....	67
4.1.2.	Node Synchronization.....	67
4.1.3.	Custom PCB Complexity.....	67
4.2.	Project Management Risks	67
4.2.1.	Scope Creep	67
4.2.2.	Time Constraints	67
4.3.	Organizational Risks.....	68
4.3.1.	App and Website Development	68
4.4.	External Risks	68
4.4.1.	Budget Constraints.....	68
4.4.2.	Battery Handling and Safety	68
4.5.	Risk Analysis Report	68
4.5.1.	Overall Risk	69
4.5.2.	Risk Priority List.....	69
	APPENDICES	70
	REFERENCES	71

List of Figures

Figure 1: SDACS Functional Block Diagram	8
Figure 2: SDACS Conceptual Diagram.....	9
Figure 3: SDACS System Block Diagram.....	10
Figure 4: SDACS Recording Node Block Diagram	14
Figure 5: SDACS Server Block Diagram	16
Figure 6: Node-RED Flow Showing Data Arriving from Firmware and Outputs to CSV and SQLite + Feature Extraction	23
Figure 6: MEMS Microphone Schematic	45
Figure 7: Temperature and Humidity Sensor Schematic.....	46
Figure 8: Power LED Array Schematic	47
Figure 9: Metro Board Pinouts Schematic.....	48
Figure 10: PCB Design for Manufacturing.....	49
Figure 7: SDACS Enclosure, Revision C	63
Figure 8: SDACS Battery Port, Revision C.....	64
Figure 9: SDACS MCU Housing, Revision C.....	65
Figure 10: SDACS Lid, Revision C.....	66

List of Tables

Table 1: MuSCoW Summary Table	12
Table 2: Node-RED Inputs and Outputs.....	29
Table 3: Firmware Inputs and Outputs	51
Table 4: Risk Assessment Matrix	68

ACKNOWLEDGEMENTS

This project is possible due to the School of Engineering and Technology at Conestoga College, specifically the faculty of the Electronic Systems Engineering – Bachelor of Applied Engineering Degree program. Special thanks to Professor Dr. So-Ra Chung, Professor Dr. Reda Fayek, and Professor Dr. Michael Galle for their continued support and valuable consultation.

1. INTRODUCTION

1.1. Purpose

The purpose of this document is to define the design of various functional units of the Smart Distributed Acoustic Calibration System (SDACS) project for Capstone I and II. This document was written for evaluation by the faculty of Conestoga College, including Professor Dr. So-Ra Chung and Professor Dr. Reda Fayek.

1.2. Project Design Description

SDACS is a spatial audio calibration network designed to model the acoustics of an internal space for the purpose of defining the best locations for recording. Inside a room, a source speaker performs a frequency sweep in the form of a logarithmic ping sequence to mimic the main audio source (i.e. a voice or other instrument) while a network of microphone nodes records the frequency response via Fast Fourier Transform (FFT), reverberation times (RT60), and Sound Pressure Level (SPL) at various locations around the room. Once calibrated, SDACS gathers the measurements from every node using Wi-Fi and Bluetooth, and outputs a three-dimensional model of the room acoustics to the user on an application/website.

The Smart Distributed Acoustic Calibration System (SDACS) consists of the following major subsystems:

1.1.1. Server

The webserver will be run using a Raspberry-Pi 5 board on the local network. The Raspberry Pi-5 will host the website / application, monitor the locations of the nodes via Bluetooth, send commands to the nodes over Wi-Fi, receive information from the nodes, and ensure network synchronization between the nodes.

Additionally, the server will perform all the calculations with the received data to create the three-dimensional model of the space and send the values as coordinates to the website application using Wi-Fi.

1.1.2. Source Node

The source node will be a studio monitor (speaker) with a practically flat EQ profile. A studio monitor ensures that the audio output signal has no effect applied to it by its hardware. The purpose of this is to provide the most accurate frequency response to a general, non-specific audio source.

1.1.3. Recording Nodes

The recording nodes will use an Adafruit Metro S3 microcontroller board, a sensors board with a MEMS microphone and a temperature/humidity sensor, and a 3.7VDC single cell Li-Po battery. The microcontroller board has pre-installed 2.4 GHz Wi-Fi and Bluetooth 5.0, voltage regulation, a microSD card port, a battery life fuel gauge, and battery charging for ease-of-use. The recording nodes will use Bluetooth to communicate their relative positioning, and use Wi-Fi to send FFTs, reverberation times, sound pressure level, and battery life to the server.

1.1.4. Client (Website / Application)

The client will be a website that sends commands via Node-RED to the server. The client will receive data from the server to display using graphs for the user to view after calibration.

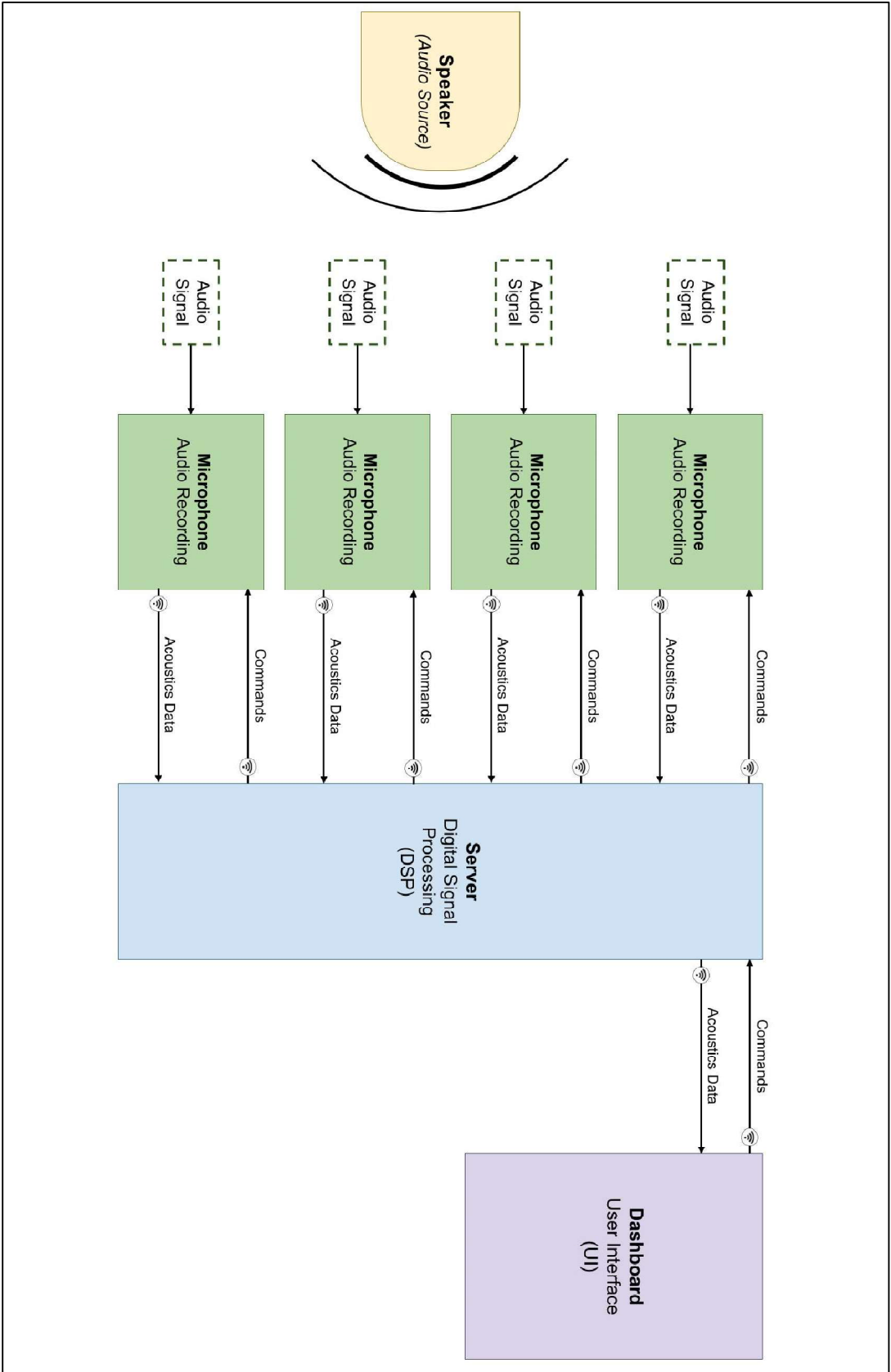


Figure 1: SDACS Functional Block Diagram

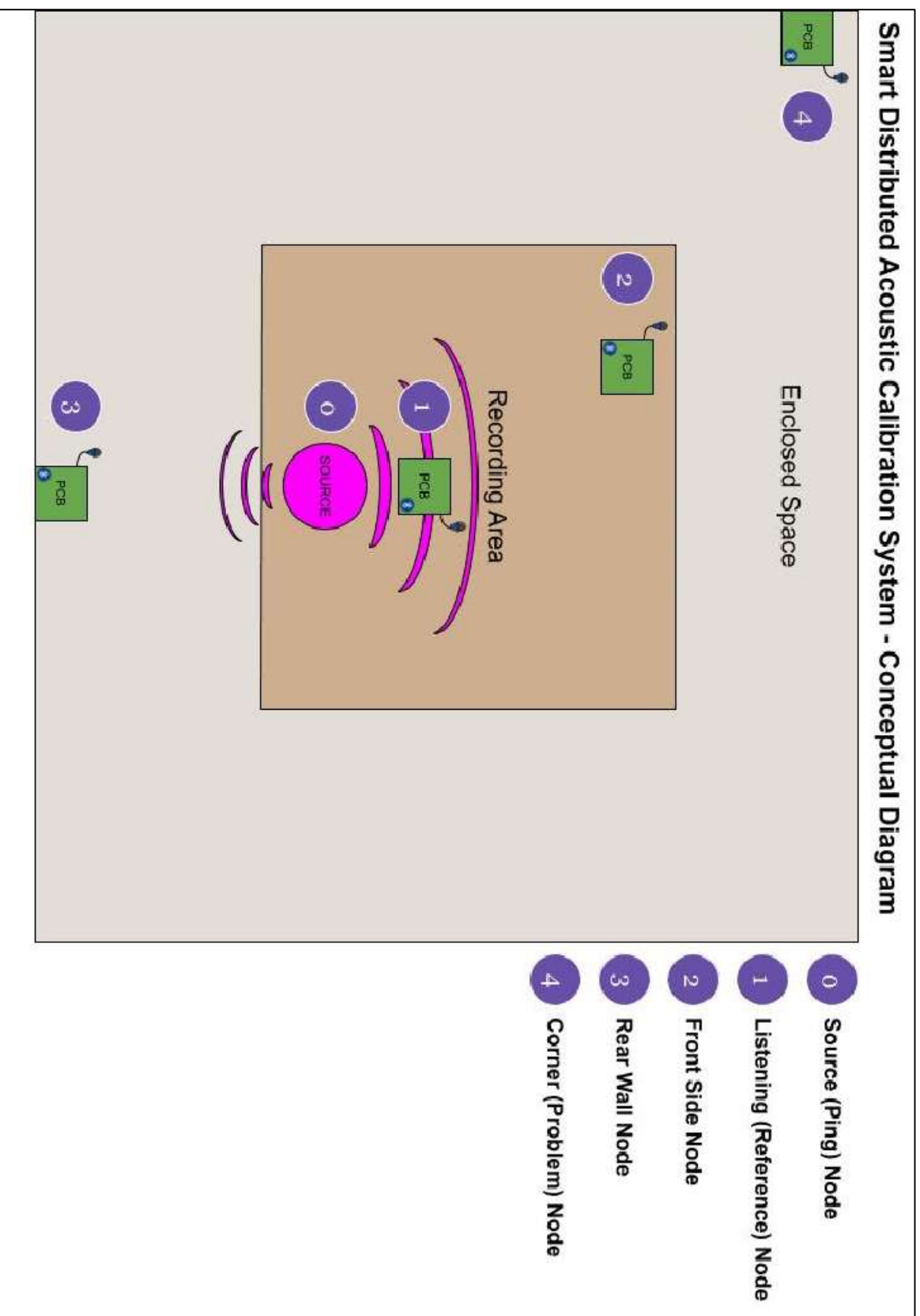


Figure 2: SDACS Conceptual Diagram

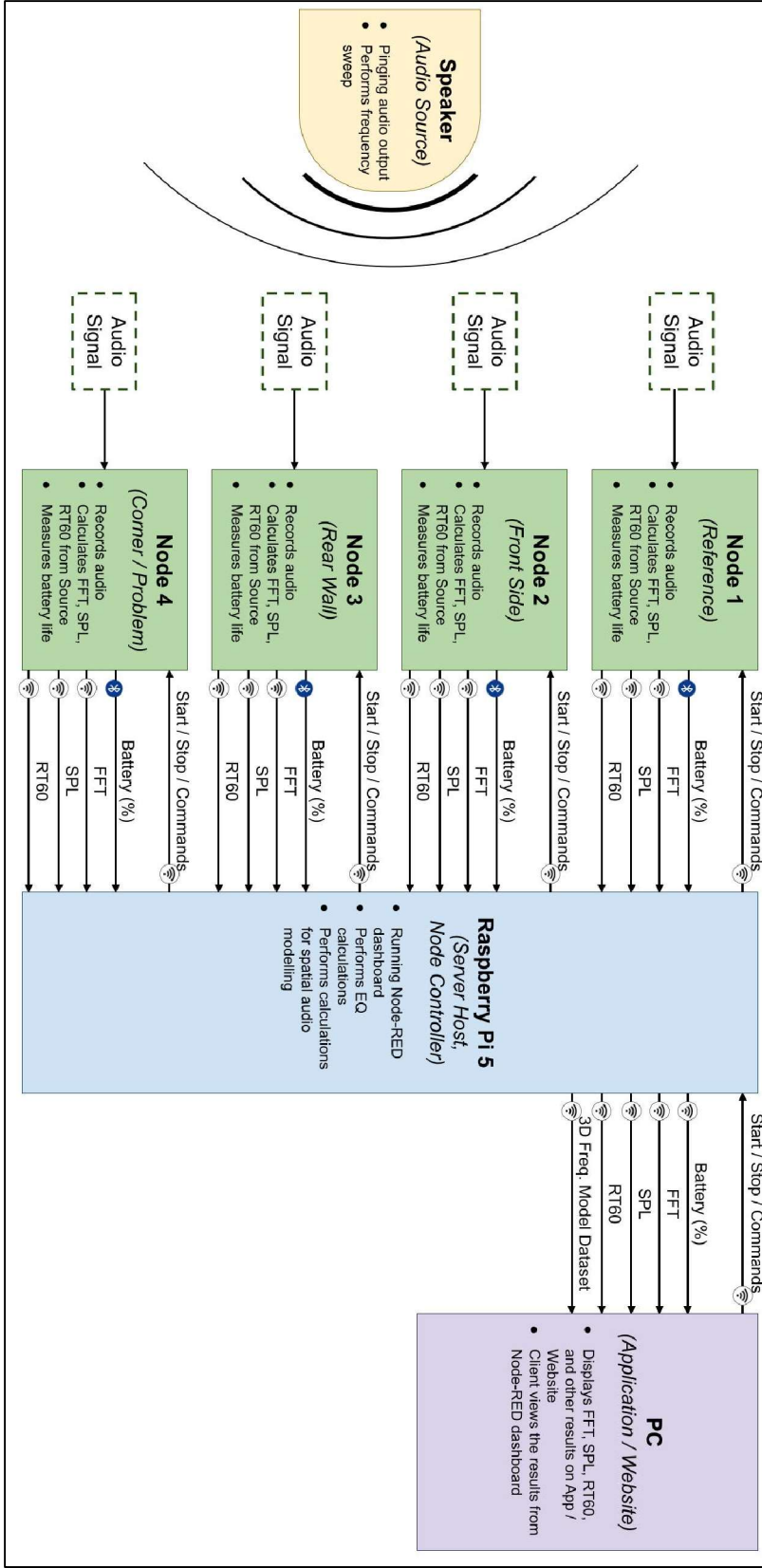


Figure 3: SDACS System Block Diagram

SDACS calibrates the three-dimensional acoustics of an indoor space in the following steps:

- i. Synchronize the nodes on the network.
- ii. Determine positioning of nodes relative to the server. Create a two-dimensional matrix describing where each node is in space.
- iii. Perform the frequency sweep. Record the time-domain audio response on the recording nodes.
- iv. Calculate FFTs, SPLs, and reverberation times (RT60) at node(s) from audio recording(s).
- v. Send results from the recording nodes to the server in Node-RED.
- vi. Create three-dimensional model of the indoor spatial acoustics using the locations of the nodes, the FFTs, the SPLs, and the reverberation times.
- vii. Send data results (co-ordinates and values) from the server to the client. Display the model(s) to the client using the website / application.

1.3. System Requirements

Table 1: MuSCoW Summary Table

Priority	Requirement
MUST	Enable simultaneous acoustic measurements from multiple indoor locations.
MUST	Utilize ESP32-based nodes with digital MEMS microphones for reliable audio capture.
MUST	Support coordinated multi-node recording for spatial comparison.
MUST	Capture, buffer, and wirelessly transmit audio data to a Raspberry Pi 5 for centralized processing.
MUST	Perform frequency-domain (FFT-based) analysis on recorded data.
MUST	Provide a web-based dashboard to display measurement results.
MUST	Demonstrate a functional prototype in a representative indoor environment.
MUST	Execute structured verification and validation testing to confirm system performance.
SHOULD	Allow remote configuration and control of measurement nodes.
SHOULD	Store measurement data for post-processing and session comparison.
SHOULD	Implement automated acoustic metrics such as RT60 estimation.
COULD	Include environmental sensing (e.g., temperature, humidity).
COULD	Perform limited on-node signal processing to reduce data transmission load.
COULD	Support scalability to additional nodes and third-party tool integration.
WON'T	Perform professional audio production or replace commercial acoustic systems.
WON'T	Include active acoustic treatment hardware or support industrial-scale deployment.

2. SYSTEM DESIGN

2.1. Functional Units

2.1.1. Recording Nodes

The recording nodes perform the following main functions:

- i. Records audio metrics, temperature and humidity, and battery life locally.
- ii. Stores all measured data on mounted SD Card.
- iii. Sends FFT, RT60, SPL, temperature/humidity and battery life to Server via Wi-Fi.

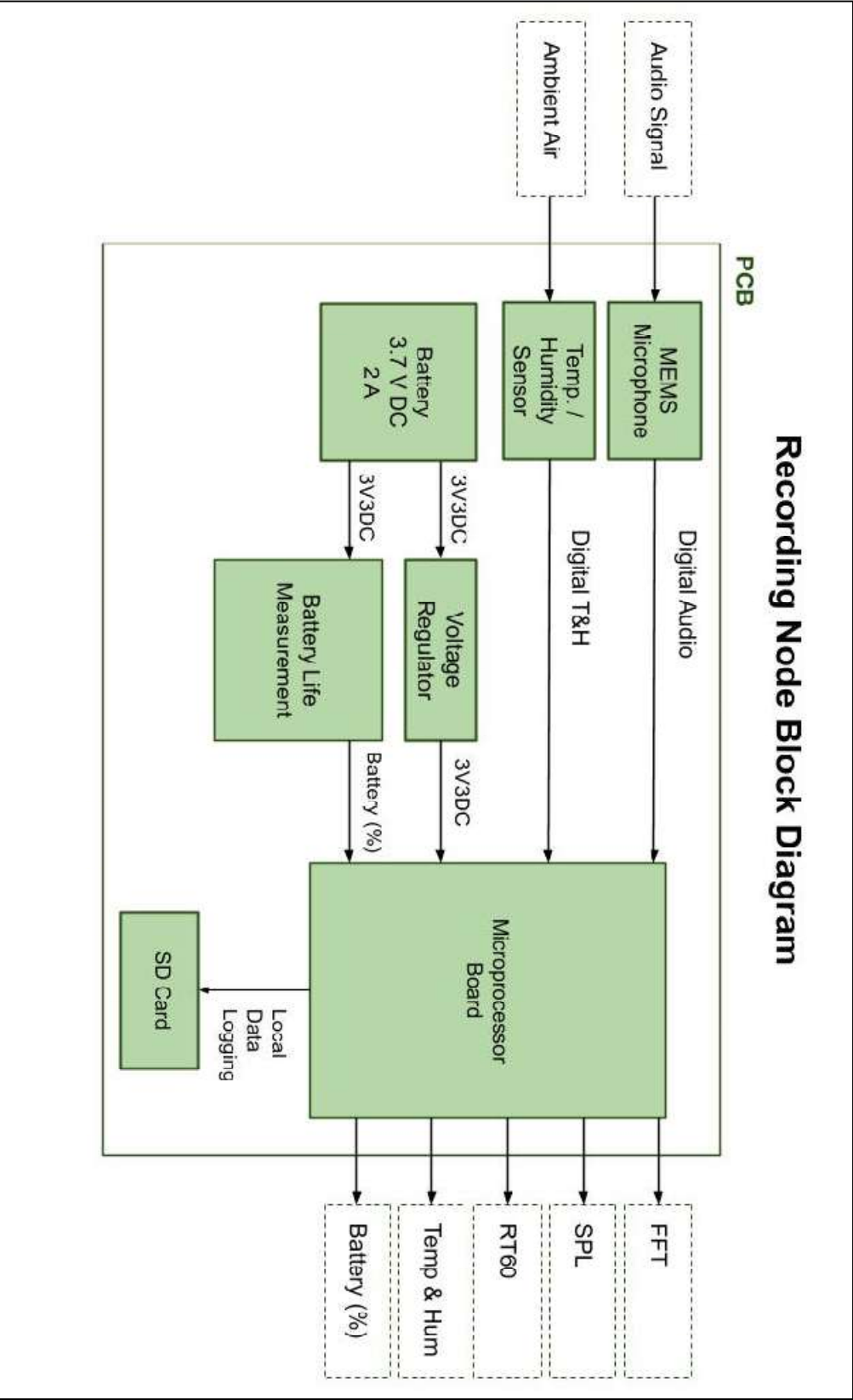


Figure 4: SDACS Recording Node Block Diagram

2.1.2. *Server*

The server performs the following main functions:

- i. Hosts Node-RED at <broker_ip>:port, and subscribes to recording nodes' MQTT topics
- ii. Synchronizes the recording nodes to initiate simultaneous capture
- iii. Sends commands including firmware updates and capture initialization to the recording nodes
- iv. Receives data (i.e. RMS, FFT, temperature and humidity), heartbeats, OTA requests, etc. from recording nodes over Wi-Fi via MQTT topics
- v. Performs analysis on the data received (converts from JSON to SQLite and CSV)
- vi. Sends updated data models, statistics, etc. to the Application and MATLAB

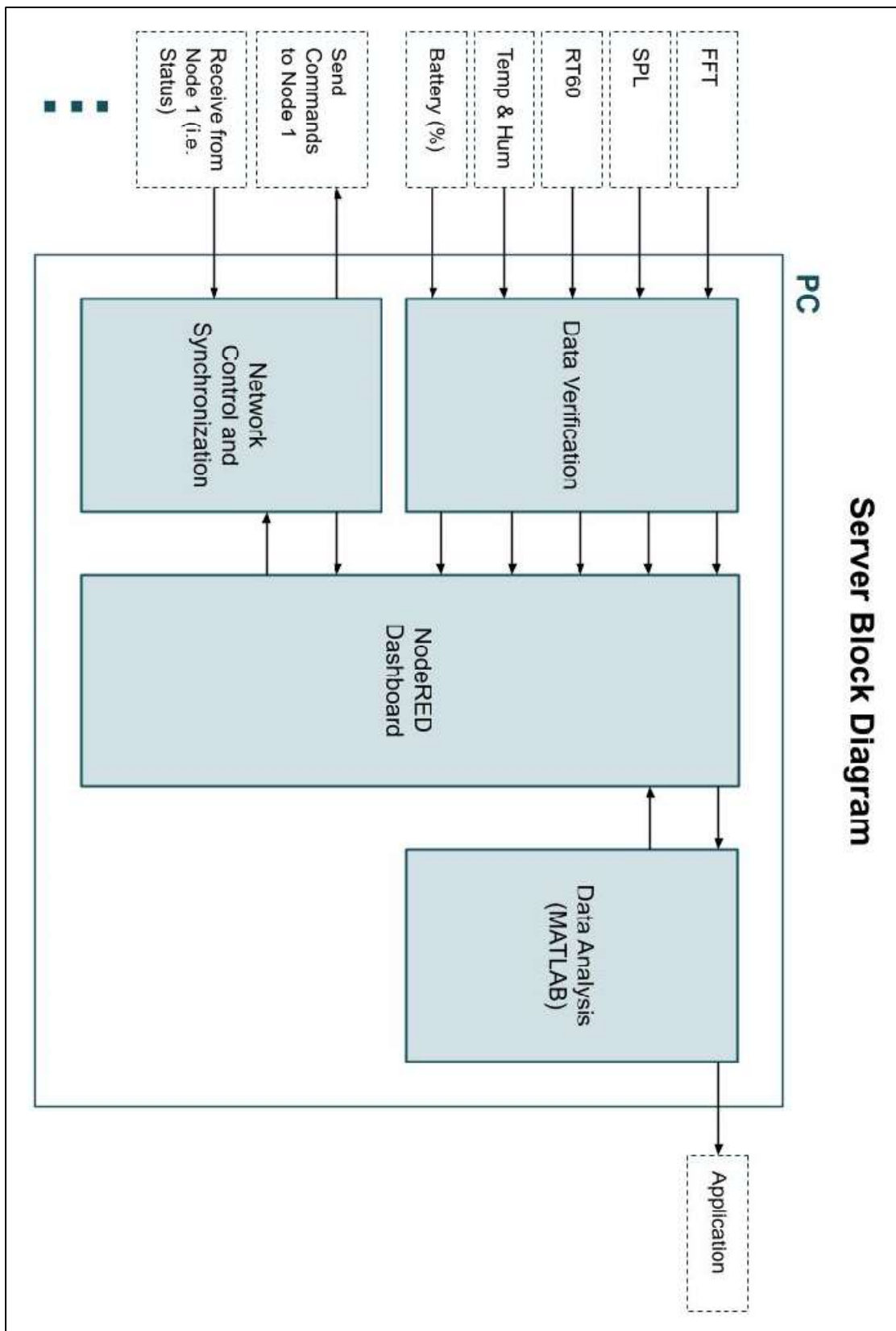


Figure 5: SDACS Server Block Diagram

2.1.3. Application (under development)

The application performs the following functions:

- i. User Interface
- ii. Initiates firmware updates
- iii. Sends user commands to the Server
- iv. Displays results of acoustic analysis using graphs, tables, etc.

2.2. Test Plans[40][42]

The SDACS system was validated through a combination of calibration testing and function system testing, targeting both individual firmware branches and overall system integration.

The system was designed to verify:

- i. Acoustic Measurement Accuracy (1kHz Calibration)
- ii. Sensor Functionality (Node-RED & Serial Monitor)
- iii. Data Transmission Reliability (Node-RED)
- iv. Data Logging Integrity (SD Card & CSV)

Testing was Divided into Two Main Categories:

- 1) Calibration and Validation Testing (1kHz Calibration)
- 2) Functional System Testing (Serial Monitor – Firmware Testing with Node-RED)

2.2.1. Recording Nodes (Summary of both Test Plans)

1) Calibration and Validation Testing

A calibration procedure was developed to establish the relationship between the digital microphone output (dBFS) and sound pressure level (SPL). This calibration was performed to determine a calibration constant that allows conversion of measured digital audio levels into real-world acoustic measurements.

The procedure was as follows:

- i. A 1 kHz reference tone was generated using a controlled audio source.
- ii. The SDACS node and a Class 2 SPL meter (Extech 407730) were positioned in the same acoustic field.
- iii. Audio data was recorded over a 10–20 second measurement window.
- iv. RMS and dBFS values were computed from firmware output.
- v. SPL readings were recorded from the reference meter.

The calibration offset was calculated as:

$$\text{SPL} = \text{dBFS} + K$$

Where K represents the calibration (offset) constant.

Validation Criteria

- i. SPL estimation within acceptable tolerance
- ii. Successful communication and system initialization

Results

- i. Average calibration constant: $K \approx 96.99 \text{ dB}$
- ii. SPL estimation accuracy: within approximately $\pm 2\text{--}3 \text{ dB}$ of reference meter

2) Functional System Testing

Functional testing was conducted to verify correct operation of all major system components and their integration. This functional testing was performed to verify the following functional units:

- i. MQTT / Node-RED
- ii. Acoustic data acquisition and processing
- iii. Temperature and humidity sensing
- iv. SD card data logging

The procedure was as follows:

- i. Configure ESP32-S3 (MQTT Client) and Raspberry Pi broker (Server) on same network.
- ii. Verify IP assignment and MQTT connection via serial output. Confirm message transmission to Node-RED.
- iii. Generate known SPL test signal (~94 dB).
- iv. Capture audio using I2S microphone.
- v. Compare measured values with reference meter.
- vi. Compare temperature readings with external thermometer.
- vii. Confirm SD card initialization.
- viii. Extract SD card and inspect file structure.
- ix. Validate CSV file contents.

Validation Criteria:

- i. Successful Wi-Fi connection and MQTT broker connection established with data visible in Node-RED.
- ii. Audio measurements within ± 4 dB
- iii. Temperature readings within $\pm 4^{\circ}\text{C}$
- iv. Non-zero, stable sensor output
- v. SD Card detected, with proper file structure generated on SD Card
- vi. CSV files on SD Card contain valid, timestamped data.

Results:

- i. All communication tests passed successfully
- ii. Data Acquisition and Processing was successful for both SPL and temperature readings
- iii. SPL measurements: Pass (± 3.2 dB error)
- iv. Temperature measurements: Marginal / inconsistent, did not meet the accuracy criteria

3) System Level Validation Summary:

Test Category	Result
Acoustic Calibration	Pass
MQTT Communication	Pass
Audio Processing	Pass
Temperature Sensing	Partial
SD Card Logging	Pass

4) Limitations Identified During Testing

- i. Temperature validation limited by low-accuracy reference thermometer
- ii. SPL reference meter (Class 2) introduced measurement variability
- iii. No controlled acoustic chamber used for 1kHz 94dB Calibration test

2.2.2. *Server (Node-RED Debug)*

The following image shows that Node-RED successfully collects the main features and extracts them from the firmware shown on the right side debug window:

2.2.3. Application (under development)

Planned Application Testing:

- i. Multi-node testing
- ii. Long-duration runtime testing
- iii. Stale/offline heartbeat logic
- iv. Full OTA success/failure testing with GitHub integration
- v. SQLite query/export validation
- vi. MATLAB import and plotting to 3D map
- vii. Bluetooth RSSI implementation with multiple nodes to define frequency map
- viii. Flutter UI Design of features
- ix. Flutter dashboard reading from SQLite

2.2.4. Off-the-shelf Components: Microcontroller, Adafruit Metro S3

The Adafruit Metro S3 is a development board based on the ESP32-S3 microcontroller, designed for embedded systems and IoT applications. It integrates Wi-Fi (2.4 GHz) and Bluetooth 5.0 connectivity, making it suitable for wireless communication in distributed sensor networks. The board includes built-in USB support, voltage regulation, battery charging circuitry, and a microSD card interface, allowing for both data acquisition and storage in a compact form factor.

The Metro S3 is used in the SDACS recording nodes to perform real-time acoustic signal acquisition, sensor integration, and wireless data transmission. Its processing capabilities allow for on-node signal processing, including RMS, dBFS, SPL estimation, and FFT calculations.

Key Features:

- i. ESP32-S3 Dual-Core Processor: Provides sufficient computational power for real-time signal processing and multitasking
- ii. Integrated Wi-Fi and Bluetooth: Enables wireless communication with the central server for data transmission and node synchronization
- iii. I2S Peripheral Support: Allows direct interfacing with digital MEMS microphones for accurate audio capture
- iv. I2C Interface: Supports environmental sensors such as temperature and humidity sensors on a shared bus
- v. MicroSD Card Support: Enables local storage of recorded data for redundancy and post-processing
- vi. Battery Management System: Includes onboard Li-Po charging and fuel gauge support for portable operation

Justification for Selection:

The Adafruit Metro S3 was selected due to its combination of wireless connectivity, processing capability, and integrated peripherals. Its compatibility with ESP-IDF and Arduino frameworks simplifies firmware development, while its compact design and built-in battery support make it ideal for distributed, portable sensing nodes.

2.2.5. Off-the-shelf Components: PC, Raspberry Pi 5

The Raspberry Pi 5 is a single-board computer used as the central server in the SDACS system. It provides sufficient processing power and networking capability to manage communication between distributed nodes, perform backend data processing, and host the Node-RED server and web-based application.

The Raspberry Pi 5 operates on a Linux-based operating system and supports multiple communication protocols, including Wi-Fi, Ethernet, and Bluetooth. In this project, it functions as the MQTT broker host, data processing unit, and interface between the hardware system and the user-facing application.

Key Features:

- i. Quad-Core ARM Cortex Processor: Enables efficient handling of multiple processes, including data aggregation and analysis
- ii. Wi-Fi and Ethernet Connectivity: Provides reliable network communication with distributed nodes
- iii. GPIO and Peripheral Support: Allows integration with additional hardware if required
- iv. Large Memory Options: Supports efficient handling of data logging, database management, and real-time processing
- v. Linux OS Support: Enables use of Node-RED, MQTT broker services, and database tools such as SQLite
- vi. USB and Storage Interfaces: Supports external devices and storage for data logging and system expansion

Justification for Selection:

The Raspberry Pi 5 was selected due to its balance of performance, cost, and flexibility. It is widely supported, easy to configure, and capable of running the required backend services for SDACS, including MQTT communication, Node-RED dashboards, and data storage systems. Its ability to handle multiple networked nodes simultaneously makes it suitable for centralized system coordination.

2.3. Safety, Societal and Environmental Issues

2.3.1 Safety Considerations

The design of the system incorporates several safety considerations to ensure reliable and secure operation. The use of a 3.3 V low-voltage system minimizes the risk of electrical hazards to users and reduces the likelihood of component damage due to overvoltage conditions. Additionally, current-limiting resistors are implemented for all LED indicators to prevent excessive current draw and overheating.

The system is powered using a rechargeable lithium polymer battery (EEMB 3.7 V, 1100 mAh), which provides a stable and portable power source. Proper charging and protection circuitry must be used to prevent overcharging, deep discharge, and short-circuit conditions, which are critical for lithium-based batteries. The inclusion of a power switch also allows users to safely disconnect power when the device is not in use.

The use of digital sensors such as the MEMS microphone and temperature/humidity sensors reduces the need for high-voltage analog circuitry, improving overall system safety and robustness.

2.3.2 Environmental Considerations

Environmental impact was considered in the selection of components and system design. The use of a rechargeable lithium polymer battery significantly reduces electronic waste compared to disposable batteries, as it can be reused over many charge cycles. Additionally, the chosen components, including the MEMS microphone and digital sensors, are highly integrated, reducing the total number of discrete components required. This minimizes material usage and simplifies manufacturing, which contributes to a lower environmental footprint.

The system is also designed with power efficiency in mind. By using low-power components such as the ESP32-S3 microcontroller and efficient LED indicators, overall energy consumption is reduced, extending battery life and decreasing the frequency of charging.

2.3.3 Societal Considerations

The primary societal impact of this project is its application in supporting music producers and audio engineers in studio environments. Accurate measurement and analysis of sound pressure levels (SPL) and room acoustics are essential for producing high-quality audio. However, professional acoustic measurement tools can be expensive and inaccessible to many users. This system provides a cost-effective and portable solution that enables users to analyze their acoustic environment, identify sound inconsistencies, and optimize their mixing and recording spaces. By improving access to acoustic measurement tools, the system supports creativity, productivity, and quality within the music production industry.

In addition to professional studio applications, the system can also benefit educational and community environments, including learning centers and spaces designed for individuals with sensory sensitivities or cognitive challenges. In these settings, excessive noise can negatively affect concentration, comfort, and overall well-being. By providing feedback on environmental noise levels, the system can help maintain controlled and quieter environments, improving accessibility and inclusivity.

3. DESIGN OF FUNCTIONAL UNITS

3.1. Software

3.1.1. Server Design – Inputs and Outputs

Node-RED is responsible for receiving all the telemetry (sensor) data from the nodes via MQTT protocol. It processes and formats data into JSON, CSV and SQLite database content and provides real-time data through a visual dashboard interface.

Table 2: Node-RED Inputs and Outputs

Inputs / MQTT Topics	Outputs
SPL	Node-RED Debug Nodes
Temperature and Humidity	CSV Log
Battery Percentage	SQLite Database
Heartbeat (Shows the node is connected to Wi-Fi)	HTTPS Endpoint for Firmware OTA (under development)

3.1.2. Node-RED Process Flow

See figure 6 for an image depicting the final Node-RED flow which contains the following node components:

1. MQTT Input (Subscribe)



```
[
  {
    "id": "mqtt_in_features",
    "type": "mqtt in",
    "name": "Node01 Features",
    "topic": "sdacs/node/node01/features",
    "datatype": "auto",
    "broker": "mqtt_broker",
    "x": 150,
    "y": 100
  },
  {
    "id": "mqtt_in_heartbeat",
    "type": "mqtt in",
    "name": "Heartbeat",
    "topic": "sdacs/node/node01/heartbeat",
    "datatype": "auto",
    "broker": "mqtt_broker",
    "x": 150,
    "y": 160
  },
  {
    "id": "mqtt_in_battery",
    "type": "mqtt in",
    "name": "Battery",
    "topic": "sdacs/node/node01/battery",
    "datatype": "auto",
    "broker": "mqtt_broker",
    "x": 150,
    "y": 220
  }
]
```

2. Parse (Convert) to JSON / decoding (SS)



```
// Node-RED Function Node: JSON Parse
if (typeof msg.payload === "string") {
  try {
    msg.payload = JSON.parse(msg.payload);
  } catch (e) {
    node.warn("Invalid JSON");
    return null;
  }
}
return msg;
```

3. Data Normalization



```
// Normalize telemetry structure
let p = msg.payload;

msg.payload = {
  timestamp: p.timestamp || new Date().toISOString(),
  node_id: p.node_id || "node01",
  rms: p.rms || 0,
  dbfs: p.dbfs || 0,
  db_spl: p.db_spl || 0,
  f_peak_hz: p.f_peak_hz || 0,
  temp_c: p.temp_c || p.temperature || null,
  rh_percent: p.rh_percent || p.humidity || null,
  battery_soc: p.soc || null,
  raw: p
};

return msg;
```

4. Route and Export

a. SQLite Database



```
msg.topic = `
INSERT INTO telemetry
(timestamp, node_id, rms, dbfs, db_spl, f_peak_hz, temp_c,
rh_percent, battery_soc)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
`;

msg.payload = [
  msg.payload.timestamp,
  msg.payload.node_id,
  msg.payload.rms,
  msg.payload.dbfs,
  msg.payload.db_spl,
  msg.payload.f_peak_hz,
  msg.payload.temp_c,
  msg.payload.rh_percent,
  msg.payload.battery_soc
];

return msg;
```

Export download Database:

Export using: http://<broker_ip>:1880/sdacs/download/db



b. CSV Logging



```
// Convert JSON → CSV row
let p = msg.payload;
```

```

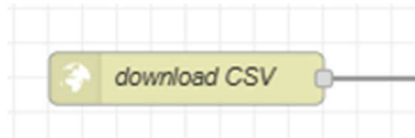
msg.payload =
`${p.timestamp},${p.node_id},${p.rms},${p.dbfs},${p.db_spl},${
p.f_peak_hz},${p.temp_c},${p.rh_percent},${p.battery_soc}\n`;

return msg;

```

CSV file path on RPi5: /home/pi/sdacs_logs/node01.csv

Export using http://<broker_ip>:1880/sdacs/download/csv



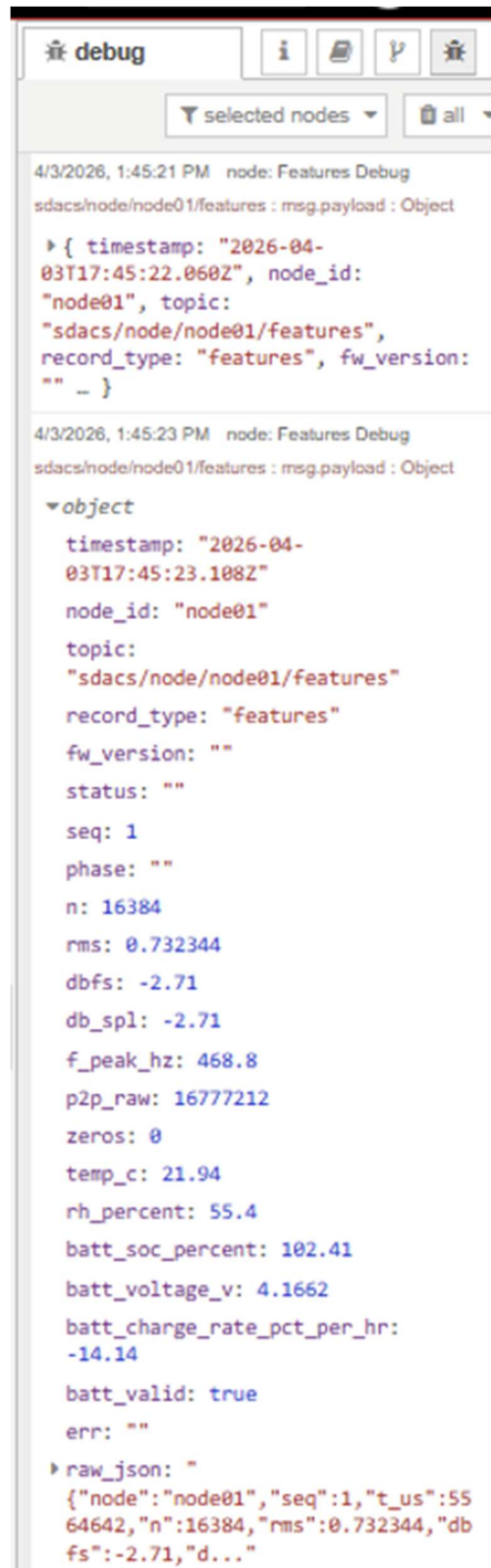
c. Debug Output (Node-RED Debug nodes):



```

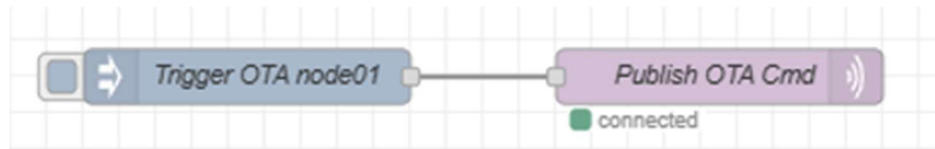
{
  "id": "debug_node",
  "type": "debug",
  "name": "Parsed Telemetry",
  "active": true,
  "tosidebar": true,
  "complete": "payload"
}

```



Shows the flow of data, good check for functioning system.

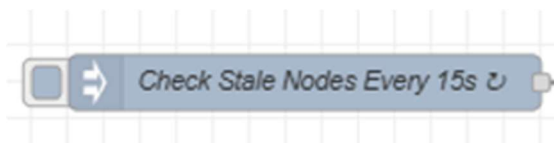
5. Cue OTA CMD from dashboard & github integration with latest firmware



```
// Trigger OTA update
msg.topic = "sdacs/node/node01/cmd";
msg.payload = {
  action: "ota_update",
  url: "https://github.com/yourrepo/firmware.bin"
};

return msg;
```

6. Stale-Nodes (Heartbeat)



```
// Node-RED Function: Track last heartbeat per node

let nodeId = msg.payload.node_id || "node01";
let now = Date.now();

// Get stored heartbeat table
let nodes = flow.get("heartbeat_nodes") || {};

// Update last seen timestamp
nodes[nodeId] = now;

flow.set("heartbeat_nodes", nodes);

return msg;
```



```
// Node-RED Function: Detect stale nodes

let timeout_ms = 10000; // 10 seconds timeout
let now = Date.now();
```

```
let nodes = flow.get("heartbeat_nodes") || {};  
let status = [];  
  
for (let nodeId in nodes) {  
  let lastSeen = nodes[nodeId];  
  let age = now - lastSeen;  
  
  status.push({  
    node_id: nodeId,  
    status: (age > timeout_ms) ? "OFFLINE" : "ONLINE",  
    last_seen_ms_ago: age  
  });  
}  
  
msg.payload = status;  
return msg;
```

3.1.3. Server Design – Investigation and Decisions Made

1) Why Node Red was Chosen:

1. Rapid prototyping of IoT devices
2. Real-Time MQTT integration
3. Visual, user friendly modular flow design
4. Simpler than a custom server

2) Why Node-RED with SDACS:

1. Easily Scalable for Multiple Nodes (Devices)
2. Easy integration with Raspberry Pi Broker

3) Key Features Implemented:

1. Acoustic Measurements (RMS, dBFS, db_SPL, FFT)
2. Multi-node compatibility sdacs/node/<node_id>
3. Data logging for post processing in MATLAB
4. Temp/Humidity monitoring
5. Heartbeat showing Wi-Fi connectivity after flash, stale when disconnected
6. Battery Statistics

4) Limitations and Future Work:

1. No database optimization with Flutter app
2. No HTTPS OTA updates or remote activation implemented
3. Dashboard is not optimized for mobile UI yet

3.1.4. MATLAB Data Analysis (under development)

MATLAB is an application used for statistics, modelling, and other signal processing applications. MATLAB contains the following libraries and/or mathematical techniques suited for the 3-D modelling of acoustics for SDACS:

1. 2-D Fast Fourier Transform

The MATLAB “fft2” function operates under the assumption of a fully sampled, uniformly spaced 2-D signal and computes its discrete Fourier based on the available samples. To model the entire room, the 2-D FFT must be used with the locational data available for the nodes to determine the (incomplete) acoustics of the room.

2. Flexible Statistics Data Analysis (FSDA) Toolbox

The FSDA (Flexible Statistics Data Analysis) toolbox introduces a statistical framework centered on robust estimation techniques designed to operate effectively in the presence of incomplete, noisy, or contaminated data. Its methodology emphasizes identifying stable underlying structures through approaches such as robust regression and forward search, where models are iteratively built from small subsets of the data and expanded while monitoring consistency. In the context of sparse Fourier data, this corresponds to treating the available coefficients as partial observations of a broader model and using robust statistical procedures to estimate the parameters governing the full spectral or spatial representation, while reducing sensitivity to outliers or irregular sampling.

3. Gaussian Distribution

A Gaussian (normal) distribution model assumes that data are independent and symmetrically distributed about a mean, fully characterized by a mean and variance, with a unimodal, bell-shaped structure and no skewness or heavy tails. The primary advantages of this model are its analytical tractability, simple parameterization, and well-defined probabilistic properties, which enable efficient estimation, closed-form solutions, and straightforward confidence interval interpretation. However, its limitations arise from these same assumptions: it cannot capture skewed distributions, multimodal structures, or heavy-tailed behavior, and may perform poorly when real data deviate significantly from normality, particularly in the presence of outliers or non-symmetric patterns.

3.2. Hardware

The hardware system consists of a compact, battery-powered embedded platform designed for distributed acoustic data acquisition, environmental sensing, and wireless communication. Each SDACS recording node integrates sensing, processing, storage, and communication hardware into a single portable unit capable of synchronized operation across a network.

3.2.1 System Overview

Each recording node consists of the following primary hardware components:

- i. Microcontroller – Adafruit Metro S3 (ESP32-S3)
- ii. Digital MEMS microphone – ICS-43434 MEMS Microphone (I2S interface)
- iii. Temperature and humidity sensor – Sensirion SHT40 (I2C interface)
- iv. Battery system – 3.7V Li-Po battery with MAX17048 Fuel Gauge IC
- v. MicroSD card interface for local data storage
- vi. Custom-designed PCB for system integration
- vii. 3D-printed enclosure for mechanical protection and portability

These components form a self-contained sensing node capable of collecting, processing, and transmitting acoustic and environmental data.

3.2.2 Hardware Architecture

The hardware architecture is centered around the ESP32-S3 microcontroller, which manages all peripheral communication and processing tasks.

Communication Interfaces:

- i. I2S (Inter-IC Sound): Dedicated high-speed interface for digital audio acquisition from the MEMS microphone
- ii. I2C (Inter-Integrated Circuit): Shared bus for environmental sensor and fuel gauge communication
- iii. SPI (SD Interface): Used for microSD card data logging
- iv. Wi-Fi / Bluetooth: Integrated wireless communication for node synchronization and data transmission

System Interconnection Overview:

- i. The MEMS microphone connects directly to the ESP32-S3 via I2S (BCLK, WS, SD lines)
- ii. The SHT40 sensor and MAX17048 fuel gauge share the same I2C bus (SDA, SCL)
- iii. The microSD card communicates via SPI for high-speed data logging
- iv. Power is distributed from the Li-Po battery through onboard regulation circuitry

3.2.3 Digital MEMS Microphone, ICS-43434

The ICS-43434 is a high-performance digital MEMS microphone designed for precision audio capture. It outputs a digital pulse-density modulated (PDM) signal that is converted internally to I2S-compatible data for direct interfacing with the ESP32-S3.

Key Characteristics:

- i. High signal-to-noise ratio (~64 dBA) for accurate acoustic measurements
- ii. Wide frequency response suitable for audio analysis
- iii. Low distortion and stable sensitivity
- iv. Digital output reduces susceptibility to electromagnetic interference

Engineering Justification:

The use of a digital I2S microphone eliminates the need for analog front-end circuitry (e.g., amplifiers, ADCs), significantly simplifying PCB design while improving noise immunity. This is critical for accurate FFT and SPL calculations in electrically noisy embedded environments.

3.2.4 Temperature and Humidity Sensor, Sensirion SHT41

The SHT41 is a high-precision digital sensor used to measure ambient temperature and relative humidity.

Key Characteristics:

- i. High accuracy ($\pm 0.2^{\circ}\text{C}$ temperature, $\pm 1.8\%$ RH typical)
- ii. Fast response time
- iii. Fully calibrated digital output
- iv. Low power consumption

Engineering Justification:

Environmental conditions such as temperature and humidity directly affect sound propagation and air density. Including the SHT40 enables contextual calibration of acoustic measurements and supports future compensation algorithms.

3.2.5 LiPo Battery, 3.7V with MAX17048 Fuel Gauge

Each node is powered by a 3.7V lithium-polymer (Li-Po) battery, enabling fully wireless and portable deployment. The MAX17048 Fuel Gauge IC monitors the battery life which can be extracted by the MCU in the form of a percentage.

Key Features:

- i. State-of-charge (SOC) estimation
- ii. Battery voltage monitoring
- iii. Charge/discharge rate tracking
- iv. I2C communication interface

Engineering Justification:

Accurate battery monitoring is critical for distributed systems to prevent node failure during measurement cycles. The MAX17048 enables predictive monitoring and reporting of battery status to the central server.

3.2.6 MicroSD Card

Each node includes a microSD card for local data storage.

Key Features:

- i. Stores raw and processed acoustic data
- ii. Supports CSV and structured logging formats
- iii. Provides redundancy in case of network failure

Engineering Justification:

Local storage ensures no data loss during communication interruptions and enables offline analysis for debugging and validation.

3.2.7 Custom PCB Design

A custom PCB was designed to integrate all system components into a compact and reliable hardware platform.

PCB Functions:

- i. Distributes power from the battery to all components
- ii. Routes communication signals (I2S, I2C, SPI)
- iii. Provides mechanical mounting for sensors and connectors

Design Features:

- i. Dedicated I2S trace routing for microphone signal integrity
- ii. Shared I2C bus with proper pull-up resistors
- iii. Decoupling capacitors placed near power pins for noise reduction
- iv. Compact layout optimized for enclosure constraints

Engineering Considerations:

- i. Signal Integrity: Careful routing of high-speed I2S lines to minimize noise and jitter
- ii. Grounding Strategy: Common ground plane to reduce electromagnetic interference
- iii. Power Stability: Decoupling and filtering to ensure stable operation under dynamic loads
- iv. Modularity: Header pins included for debugging and expansion

3.2.8 Schematics

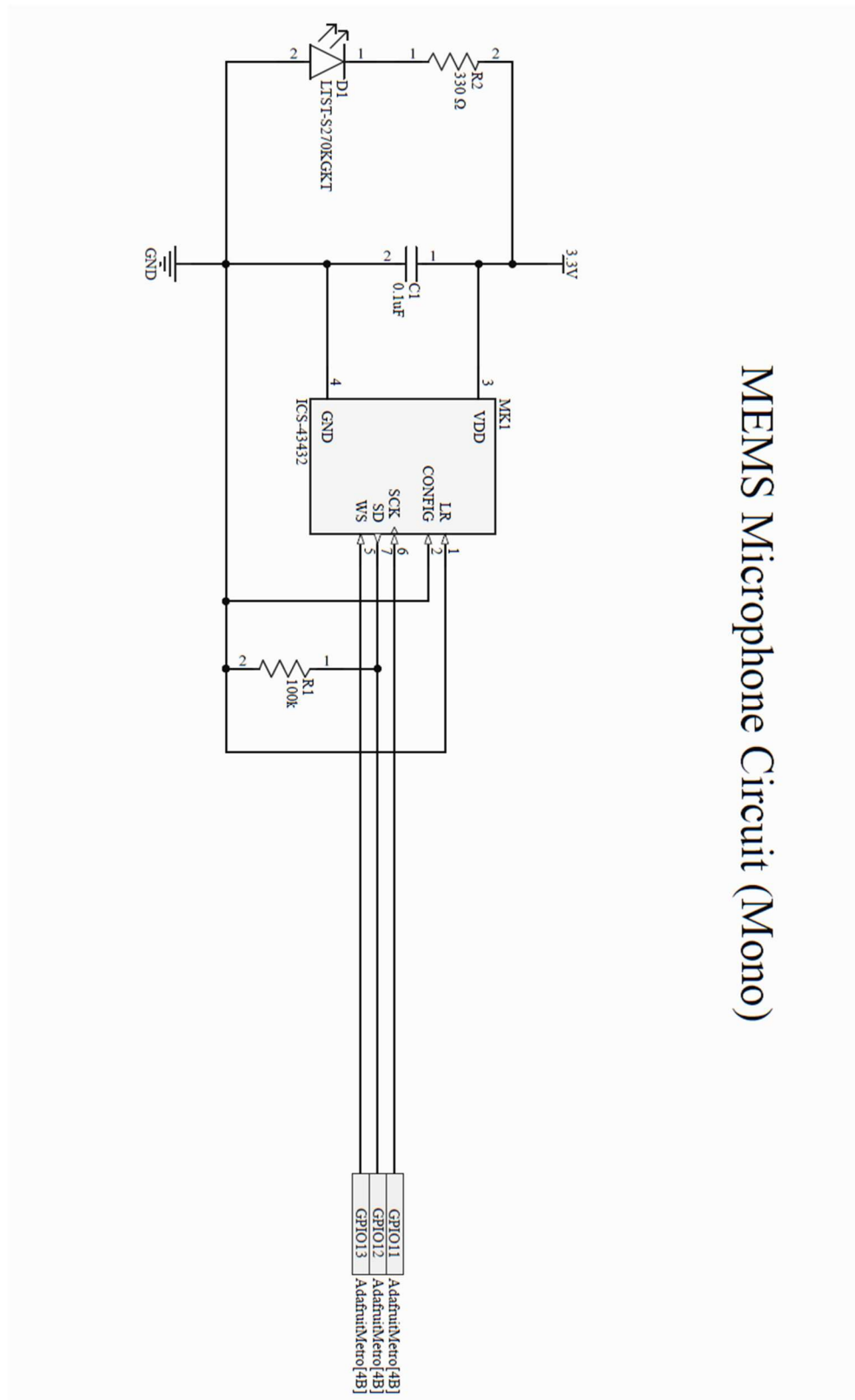


Figure 7: MEMS Microphone Schematic

Temperature/Humidity Sensor

Figure 8: Temperature and Humidity Sensor Schematic

Power LEDs (Battery Life)

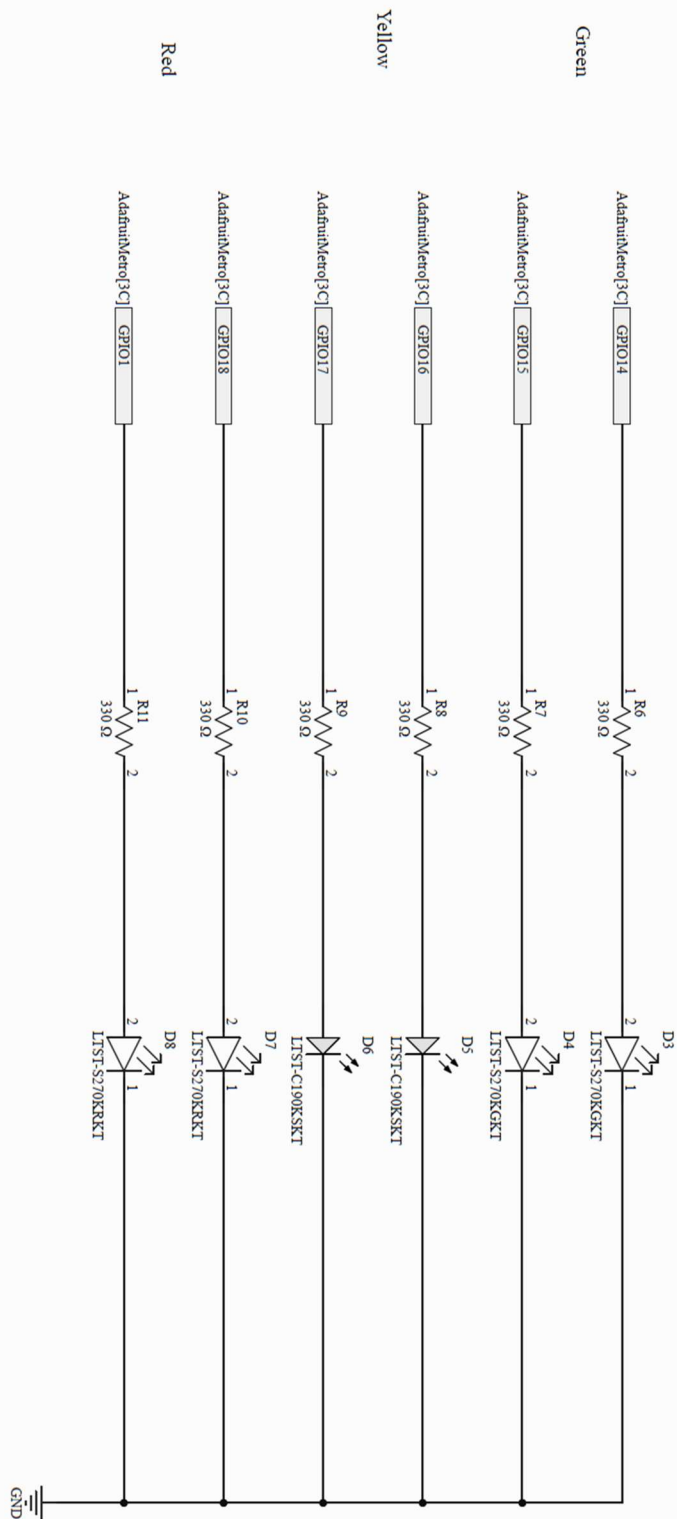


Figure 9: Power LED Array Schematic

Metro Board Pinout (Headers)

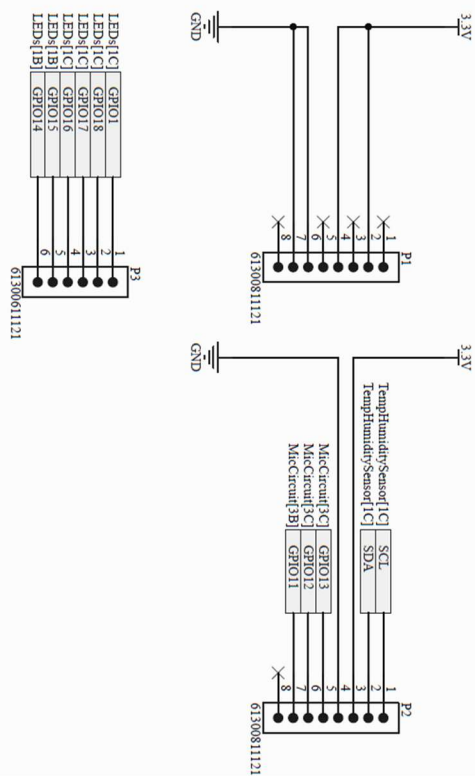
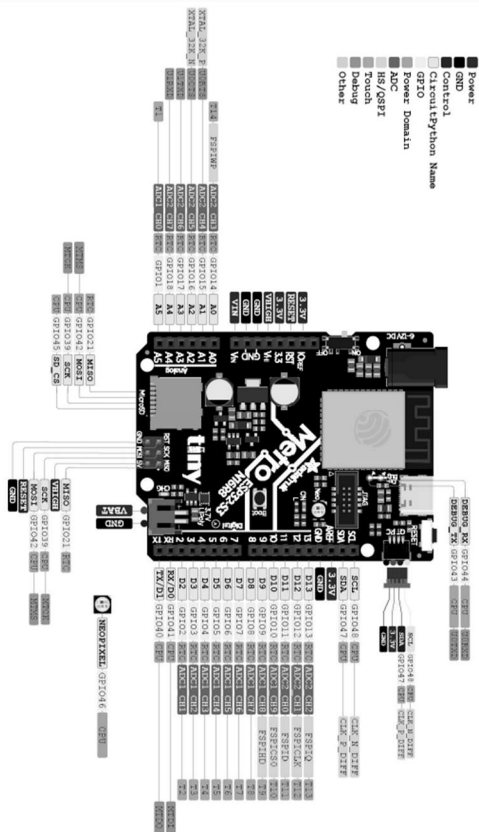


Figure 10: Metro Board Pinouts Schematic

3.2.9 PCB Layout

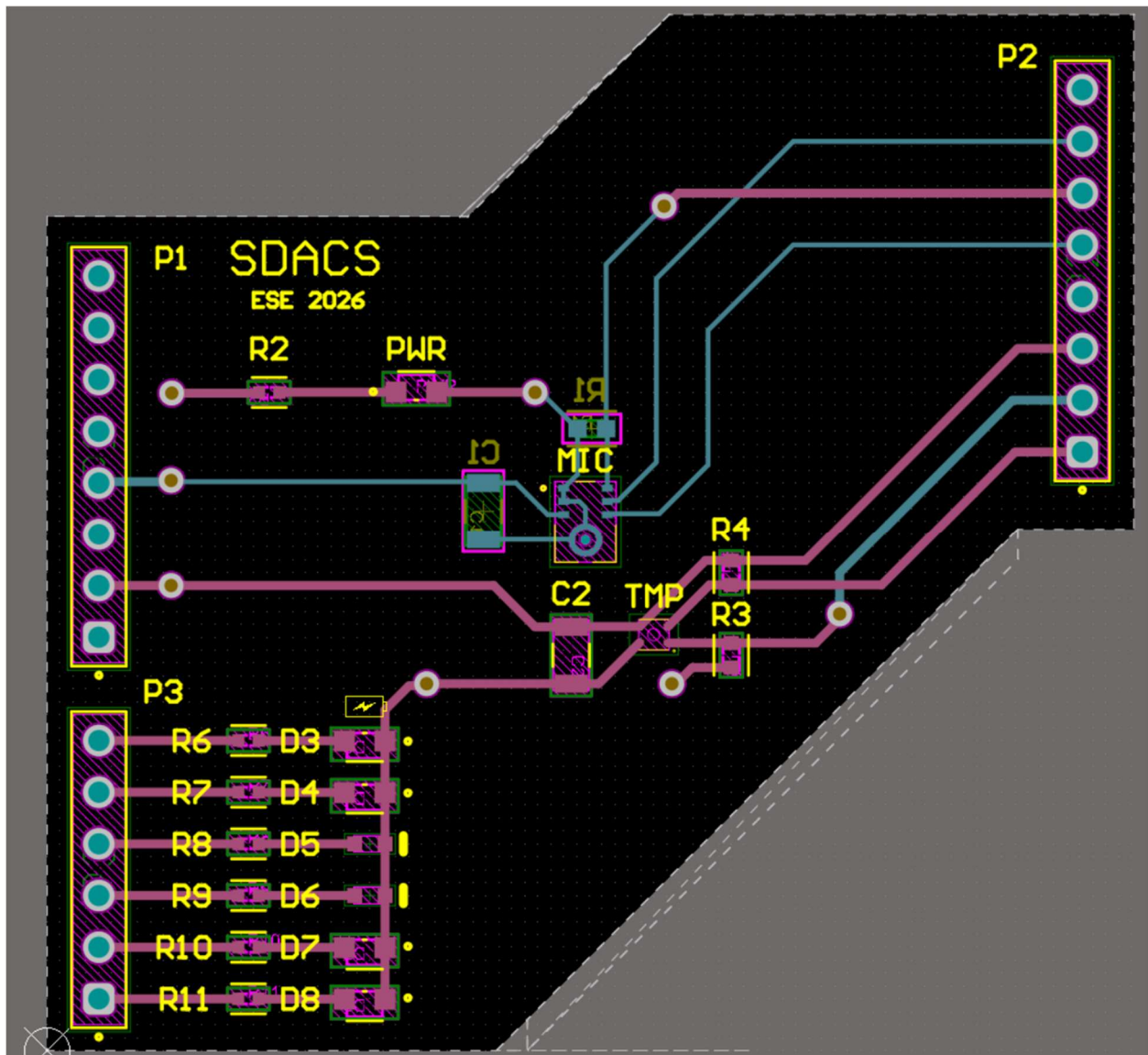


Figure 11: PCB Design for Manufacturing

3.3. Firmware

The embedded firmware system is responsible for real-time data acquisition, signal processing, sensor integration, and wireless communication across distributed nodes. Each node operates independently while contributing to a synchronized system through MQTT-Based telemetry.

3.3.1 Firmware Design

1. Acoustic signal measurement acquisition using an I2S MEMS microphone
2. Local feature extraction including RMS, dBFS, dB_SPL & FFT-based peak frequency

```
// (From audio_input.c)
i2s_read(I2S_NUM_0, samples, sizeof(samples), &bytes_read, portMAX_DELAY);
```

3. Environmental sensing via a shared I2C bus

```
// (From shared_i2c_bus.c)
i2c_config_t conf = {
    .mode = I2C_MODE_MASTER,
    .sda_io_num = cfg->sda_gpio,
    .scl_io_num = cfg->scl_gpio,
}; // Shows unified I2C Bus
```

4. Battery monitoring using MAX17048 fuel gauge

```
// (From fuel_gauge.c showing state of charge SOC, and Battery Voltage)
err = max17048_read_reg16(port, addr, MAX17048_REG_VCELL, &raw_vcell);
err = max17048_read_reg16(port, addr, MAX17048_REG_SOC, &raw_soc);

out->voltage_v = (float)raw_vcell * 78.125e-6f;
out->soc_percent = (float)raw_soc / 256.0f;
```

5. Wi-Fi and MQTT communication with Broker (Raspberry Pi 5) hosting Node-RED backend

```
// (Shows app_main.c publishing to a topic)
esp_mqtt_client_publish(client,
    "sdacs/node/node01/features",
    json_string, 0, 1, 0);
```

3.3.2 Inputs and Outputs

Table 3: Firmware Inputs and Outputs

Inputs	Outputs
I2S digital audio stream from MEMS microphone	RMS amplitude
I2C sensor data (temperature, humidity, fuel gauge)	dBFS / SPL
System timing signals, (sampling intervals, task scheduling)	Peak frequency (FFT)
	Temperature and Humidity
	Battery Percentage
	MQTT Topics (See Server Design)

3.3.3 Firmware Process Flow

- 1) Audio samples are continuously captured via the I2S peripheral at a fixed sampling rate (e.g., 48 kHz)

```
// I2S streaming (audio_input.c)
i2s_read(I2S_NUM_0, samples, sizeof(samples), &bytes_read, portMAX_DELAY);
```

- 2) Samples are buffered in fixed-size blocks (e.g., 512 samples)

```
// capture_task.c
static int32_t read_buf[SDACS_I2S_FRAMES_PER_READ];

err = audio_input_read_s24(
    read_buf,
    SDACS_I2S_FRAMES_PER_READ,
    &samples_read,
    SDACS_I2S_READ_TIMEOUT_MS
);
```

- 3) Each block undergoes signal processing:

- a) RMS calculation dBFS conversion /dB_SPL estimation

```
float rms = sqrtf((float)(s_fft.sum_sq / (double)s_fft.count));
float rms_norm = rms / 8388608.0f;
float dbfs = 20.0f * log10f(rms_norm + 1e-12f);

*out = (audio_metrics_t){
    .rms_norm = rms_norm,
    .dbfs = dbfs,
    .laeq_db = dbfs + cal_offset_db,
};
```

- b) FFT-based frequency analysis

```
// Prepare the FFT input buffer using normalized, windowed samples.
s_fft.fft_in[2 * i] = sample * s_fft.hann_window[i];
s_fft.fft_in[(2 * i) + 1] = 0.0f;
}
```

- 4) Temp/Humidity and fuel gauge readings are acquired periodically over the 20s capture over a shared I2C bus.

a) Temperature readings acquired:

```
//From temp_humidity.c
const uint8_t cmd[2] = { 0x24, 0x00 };
esp_err_t err = i2c_write(port, addr, cmd, sizeof(cmd));
err = i2c_read(port, addr, buf, sizeof(buf));

*temp_c      = -45.0f + 175.0f * ((float)raw_t / 65535.0f);
*rh_percent = 100.0f * ((float)raw_rh / 65535.0f);
```

b) Fuel Gauge Readings Acquired:

```
//From fuel_gauge.c
err = max17048_read_reg16(port, addr, MAX17048_REG_VCELL, &raw_vcell);
err = max17048_read_reg16(port, addr, MAX17048_REG_SOC, &raw_soc);

out->voltage_v = (float)raw_vcell * 78.125e-6f;
out->soc_percent = (float)raw_soc / 256.0f;

[additional fields omitted for brevity]

err = max17048_read_reg16(port, addr, MAX17048_REG_CRATE, &raw_crate);
out->charge_rate_percent_per_hr = (float)((int16_t)raw_crate) * 0.208f;
```

c) I2C 20s periodic Capture:

```
//From capture_task.c
while (esp_timer_get_time() < end_us) {
    ...
    if (now_us >= next_metrics_us) {
        if (temp_humidity_get_latest(&th)) {
            temp_c = th.temp_c;
            humidity = th.rh_percent;
        }
        if (fuel_gauge_get_latest(&batt)) {
            batt_soc_percent = batt.soc_percent;
            batt_voltage_v = batt.voltage_v;
            batt_charge_rate_pct_per_hr =
batt.charge_rate_percent_per_hr;
        }
        ...
        next_metrics_us += 1000000LL;
    }
}
```

```

    }
}

```

d) Shared I2C bus

```

// from shared_i2c_bus.c
i2c_config_t conf = {
    .mode = I2C_MODE_MASTER,
    .sda_io_num = cfg->sda_gpio,
    .scl_io_num = cfg->scl_gpio,
    .sda_pullup_en = GPIO_PULLUP_ENABLE,
    .scl_pullup_en = GPIO_PULLUP_ENABLE,
    .master.clk_speed = cfg->freq_hz,
};

err = i2c_driver_install(cfg->port, conf.mode, 0, 0, 0);

```

- 5) Processed data is formatted into structured JSON payloads and organized into MQTT topics.

```

// Shows a compact JSON message for the 1 Hz feature stream.
// This payload contains the key acoustic, environmental,
// and battery metrics needed by Node-RED.
static int build_features_json(char *out, size_t out_sz, const
sdacs_features_t *f)
{
    char temp_c_buf[24] = "null";
    char rh_percent_buf[24] = "null";
    char batt_soc_buf[24] = "null";
    char batt_voltage_buf[24] = "null";
    char batt_rate_buf[24] = "null";
    const char *batt_valid_str = f->batt_valid ? "true" : "false";

    .
    .
    .

    // Format the structured JSON payload sent to the dashboard backend.
    return snprintf(out, out_sz,
        "{
            \"node\": \"%s\",
            \"seq\": %u,
            \"t_us\": %\" PRIu64 \",
            \"n\": %u,

```

```

        "\"rms\":"%.6f,"
        "\"dbfs\":"%.2f,"
        "\"db_spl\":"%.2f,"
        "\"fft_peak_Hz\":"%.1f,"
        "\"f_peak_hz\":"%.1f,"
        "\"p2p_raw\":"%" PRId32 " ",
        "\"zeros\":"%d,"
        "\"temp_c\":"%s,"
        "\"rh_percent\":"%s,"
        "\"batt_soc_percent\":"%s,"
        "\"batt_voltage_v\":"%s,"
        "\"batt_charge_rate_pct_per_hr\":"%s,"
        "\"batt_valid\":"%s"
    "}",
    f->node_id,
    (unsigned)f->seq,
    (uint64_t)f->t_us,
    (unsigned)f->n,
    f->rms,
    f->dbfs,
    f->db_spl,
    f->f_peak_hz,
    f->f_peak_hz,
    f->p2p_raw,
    f->zeros,
    temp_c_buf,
    rh_percent_buf,
    batt_soc_buf,
    batt_voltage_buf,
    batt_rate_buf,
    batt_valid_str
); //formats data so it's ready for Node-RED parsing (JSON)
}

```

```

// Normalize the MQTT topic into a reusable base topic.
// This allows the firmware to derive related topics such as
// /features, /audio, and /status/heartbeat from one root path.
static void normalize_base_topic(const char *configured_topic, char *out,
size_t out_sz)
{
    if (!out || out_sz == 0) {
        return;
    }
}

```

```

out[0] = '\0';
if (!configured_topic || configured_topic[0] == '\0') {
    return;
}

strncpy(out, configured_topic, out_sz - 1);
out[out_sz - 1] = '\0';

if (strip_topic_suffix(out, "/status/heartbeat")) return;
if (strip_topic_suffix(out, "/status")) return;
if (strip_topic_suffix(out, "/features")) return;
if (strip_topic_suffix(out, "/audio")) return;
if (strip_topic_suffix(out, "/temp_humidity")) return;
}

```

```

// Include node identity in each feature packet so the backend
// can distinguish between multiple distributed sensing nodes.
strncpy(feats.node_id, state->ctx.node_id, sizeof(feats.node_id) - 1);

```

6) Data is transmitted via MQTT over Wi-Fi to the central broker

```

// Configure and start the MQTT client after Wi-Fi is available.
// A Last Will message is used so the broker can detect unexpected
// disconnects.
esp_mqtt_client_config_t mqtt_cfg = {
    .broker.address.uri = broker_uri,
    .session.last_will.topic = s_heartbeat_topic,
    .session.last_will.msg = s_lwt_payload,
    .session.last_will.qos = 1,
    .session.last_will.retain = 1,
};

s_mqtt = esp_mqtt_client_init(&mqtt_cfg);
ESP_ERROR_CHECK(esp_mqtt_client_register_event(
    s_mqtt, ESP_EVENT_ANY_ID, mqtt_event_handler, NULL));
ESP_ERROR_CHECK(esp_mqtt_client_start(s_mqtt));

```

Overall: this pipeline enables continuous, low-latency streaming of meaningful acoustic and environmental features.

3.3.4 Firmware System Architecture

1) Driver Layer

- a) I2S interface (microphone)
- b) I2C interface (sensors and fuel gauge)

```
// Initialize the hardware driver layer:  
// shared I2C for environmental sensors and fuel gauge,  
// and the audio input path for the I2S MEMS microphone.  
ESP_ERROR_CHECK(shared_i2c_bus_init(&i2c_cfg));  
ESP_ERROR_CHECK(audio_input_init());
```

2) Processing Layer

- a) Audio buffering
- b) Feature extraction (RMS, FFT, peak detection)

```
// Read a fixed-size audio block, then pass it into the  
// signal-processing pipeline for feature extraction.  
static int32_t read_buf[SDACS_I2S_FRAMES_PER_READ];  
  
err = audio_input_read_s24(  
    read_buf,  
    SDACS_I2S_FRAMES_PER_READ,  
    &samples_read,  
    SDACS_I2S_READ_TIMEOUT_MS  
);  
  
fft_metrics_accumulate_block(read_buf, samples_read);
```

3) Communication Layer

- a) Wi-Fi connectivity
- b) MQTT client handling
- c) Topic-based message publishing

```
// Start the communication subsystem and register the
// callback used to handle incoming broker-side commands.
ESP_ERROR_CHECK(wifi_mqtt_start(NULL));
ESP_ERROR_CHECK(wifi_mqtt_set_command_callback(command_dispatcher_handl
e));
```

```
// Non-blocking queue handoff keeps the real-time capture path
// separate from network transmission delays.
bool wifi_mqtt_try_send(const sdacs_features_t *f)
{
    if (!s_feat_q || !f) return false;
    return (xQueueSend(s_feat_q, f, 0) == pdTRUE);
}
```

4) Application Layer

- a) Task scheduling
- b) Data formatting
- c) System state management

```
// Build the capture context used by the application layer.
// This ties together node identity, topic structure,
// storage, and task-level runtime state.
capture_context_t ctx = {
    .storage = &s_storage,
    .node_id = SDACS_NODE_ID,
    .base_topic = base_topic,
};

ESP_ERROR_CHECK(capture_task_start(&ctx));
```

This layered architecture ensures separation of concerns and supports future expansion (e.g., additional sensors or processing algorithms).

3.3.5 Firmware Design Decisions and Justifications

1) Edge Feature Extraction vs Raw Audio Streaming

Instead of transmitting raw audio data, the firmware extracts key acoustic features locally.

- i) Reduces bandwidth requirements significantly
- ii) Enables real-time visualization and comparison
- iii) Improves system scalability for multiple nodes

```
// Extract and package compact acoustic features locally
// instead of transmitting only raw audio continuously.
sdacs_features_t feat = {0};
feat.rms = metrics.rms_norm;
feat.dbfs = metrics.dbfs;
feat.db_spl = metrics.laeq_db;
feat.f_peak_hz = metrics.fft_peak_hz;
feat.temp_c = temp_c;
feat.rh_percent = humidity;
feat.batt_soc_percent = batt_soc_percent;
```

2) Chunk-Based Streaming Instead of Full Buffering

Audio data is processed in fixed-size chunks rather than storing large buffers.

- i) Avoids memory overflow on ESP32-S3
- ii) Prevents system instability and watchdog resets
- iii) Enables continuous real-time operation

```
// Fixed-size acquisition and processing limits RAM usage
// and supports stable continuous operation on the ESP32-S3.
#define SDACS_I2S_FRAMES_PER_READ      512
#define SDACS_AUDIO_CHUNK_SAMPLES      2048
#define SDACS_FFT_SIZE                  1024
.
.
.
static int32_t read_buf[SDACS_I2S_FRAMES_PER_READ];
```

3) Unified I2C Bus for Sensors

All environmental sensors and the fuel gauge share a single I2C bus.

```
// The fuel gauge and temp/humidity sensor share the same
// I2C controller, simplifying wiring and bus management.
#define SDACS_TEMP_HUMIDITY_I2C_PORT    0
#define SDACS_FUEL_GAUGE_I2C_PORT      SDACS_TEMP_HUMIDITY_I2C_PORT
```

4) MQTT-Based Communication Architecture

MQTT was selected for lightweight, scalable communication.

- i) Supports distributed node architecture
- ii) Enables real-time data streaming
- iii) Easily integrates with Node-RED backend

```
// Use a queue to decouple real-time processing from MQTT publishing.
if (!s_feat_q) {
    s_feat_q = xQueueCreate(FEATURES_QUEUE_LEN, sizeof(sdacs_features_t));
    if (!s_feat_q) return ESP_ERR_NO_MEM;
}

[additional details removed for conciseness]

bool wifi_mqtt_try_send(const sdacs_features_t *f)
{
    if (!s_feat_q || !f) return false;
    return (xQueueSend(s_feat_q, f, 0) == pdTRUE);
} // Shows lightweight and scalable MQTT Architecture
```

5) Modular Firmware Structure

Firmware is divided into functional modules.

- i) Simplifies debugging and testing
- ii) Enables parallel development
- iii) Supports future feature additions

```
// Functional separation is enforced through individual modules
// for sensing, processing, communication, storage, and control.
#include "audio_input.h"
#include "capture_task.h"
#include "fft_metrics.h"
#include "fuel_gauge.h"
#include "shared_i2c_bus.h"
#include "temp_humidity.h"
#include "wifi_mqtt.h"
```

3.3.6 Key Features Implemented

- i) Real-time acoustic feature extraction (RMS, dBFS, dB_SPL, FFT peak)
- ii) Continuous streaming of processed data via MQTT
- iii) Multi-sensor integration (audio, temperature, humidity, battery)
- iv) Support for distributed multi-node deployment

```
// Final per-second feature packet contains acoustic,  
// environmental, and battery telemetry for multi-node streaming.  
feat.rms = metrics.rms_norm;  
feat.dbfs = metrics.dbfs;  
feat.db_spl = metrics.laeq_db;  
feat.f_peak_hz = metrics.fft_peak_hz;  
feat.temp_c = temp_c;  
feat.rh_percent = humidity;  
feat.batt_soc_percent = batt_soc_percent;  
feat.batt_voltage_v = batt_voltage_v;  
  
[additional details removed for conciseness]  
  
wifi_mqtt_try_send(&feat);
```

3.3.7 Limitations and Future Improvements

- i) OTA firmware updates
- ii) advanced DSP (e.g., RT60 estimation)
- iii) adaptive power management strategies

```
// OTA support was partially scaffolded in the firmware  
// but not fully completed in the final implemented system.  
#include "ota_manager.h"
```

3.4. Enclosure Design

The enclosure was designed to be small, portable, and easily accessible for prototyping and testing purposes. Additionally, the enclosure was designed to be as light as possible, so the material used was PLA plastic via 3-D printing. The enclosure design is still under development, but the most current revision (Revision C) is shown below:

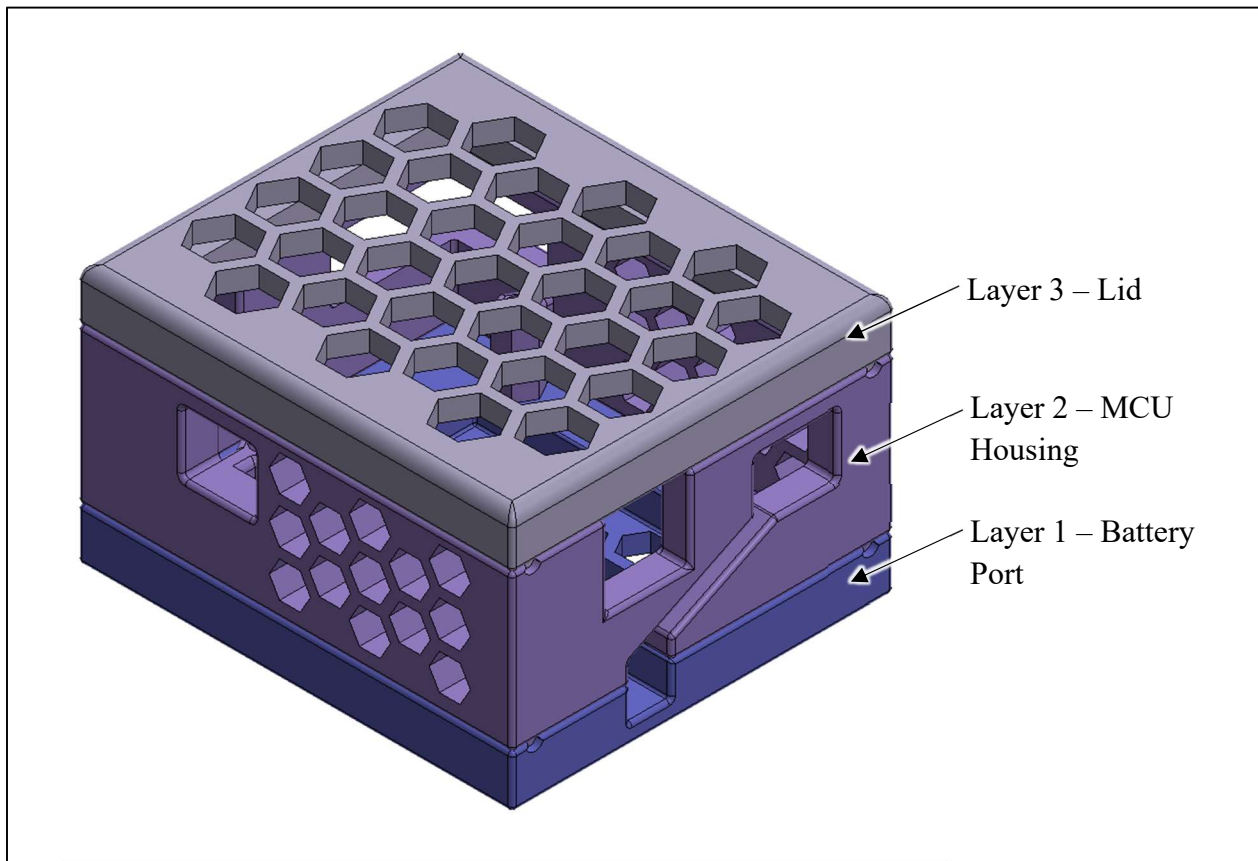


Figure 12: SDACS Enclosure, Revision C

The enclosure contains the LiPo battery, the MCU, and the custom PCB (mounted on MCU). The enclosure consists of three main components:

- Layer 1 - Battery Port
- Layer 2 - MCU Housing
- Layer 3 - Lid

3.4.1. Battery Port

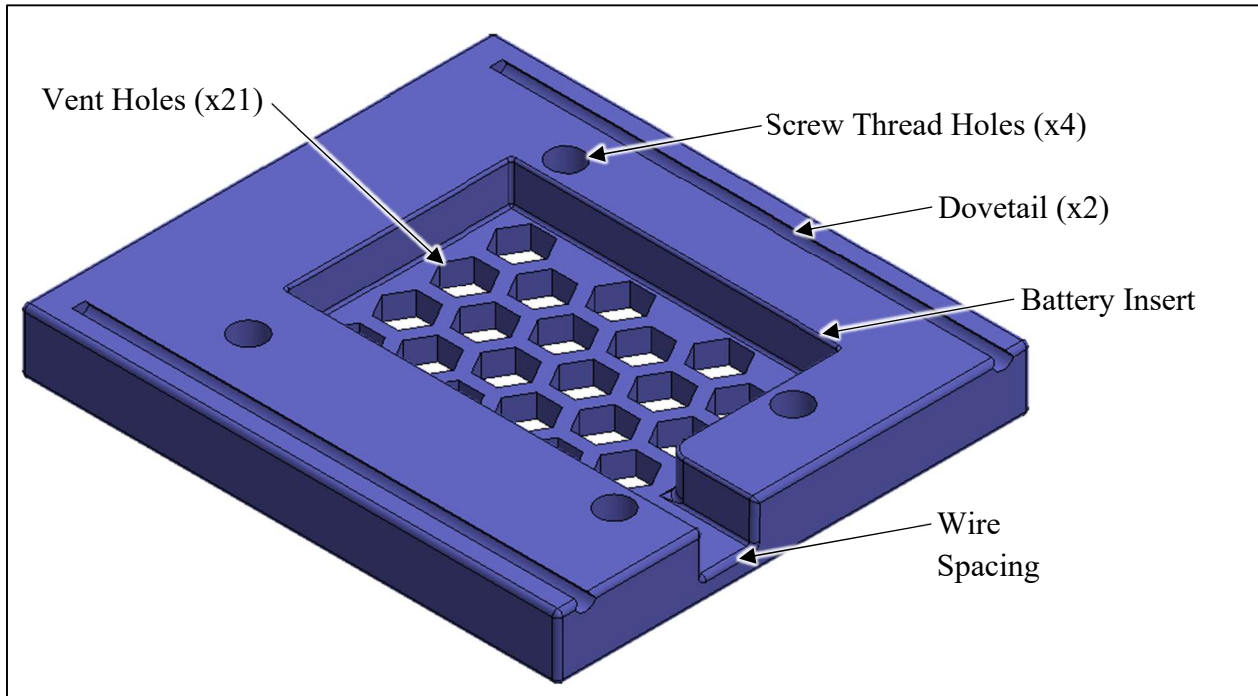


Figure 13: SDACS Battery Port, Revision C

The battery port provides housing for the LiPo battery, which powers the entire SDACS system. The battery insert has ventilation holes to prevent over-heating during operation, and a wire spacing port for the battery connector. The battery port additionally contains the screw thread holes for the standoffs, and a dovetail insert for connection to the MCU Housing layer.

3.4.2. MCU Housing

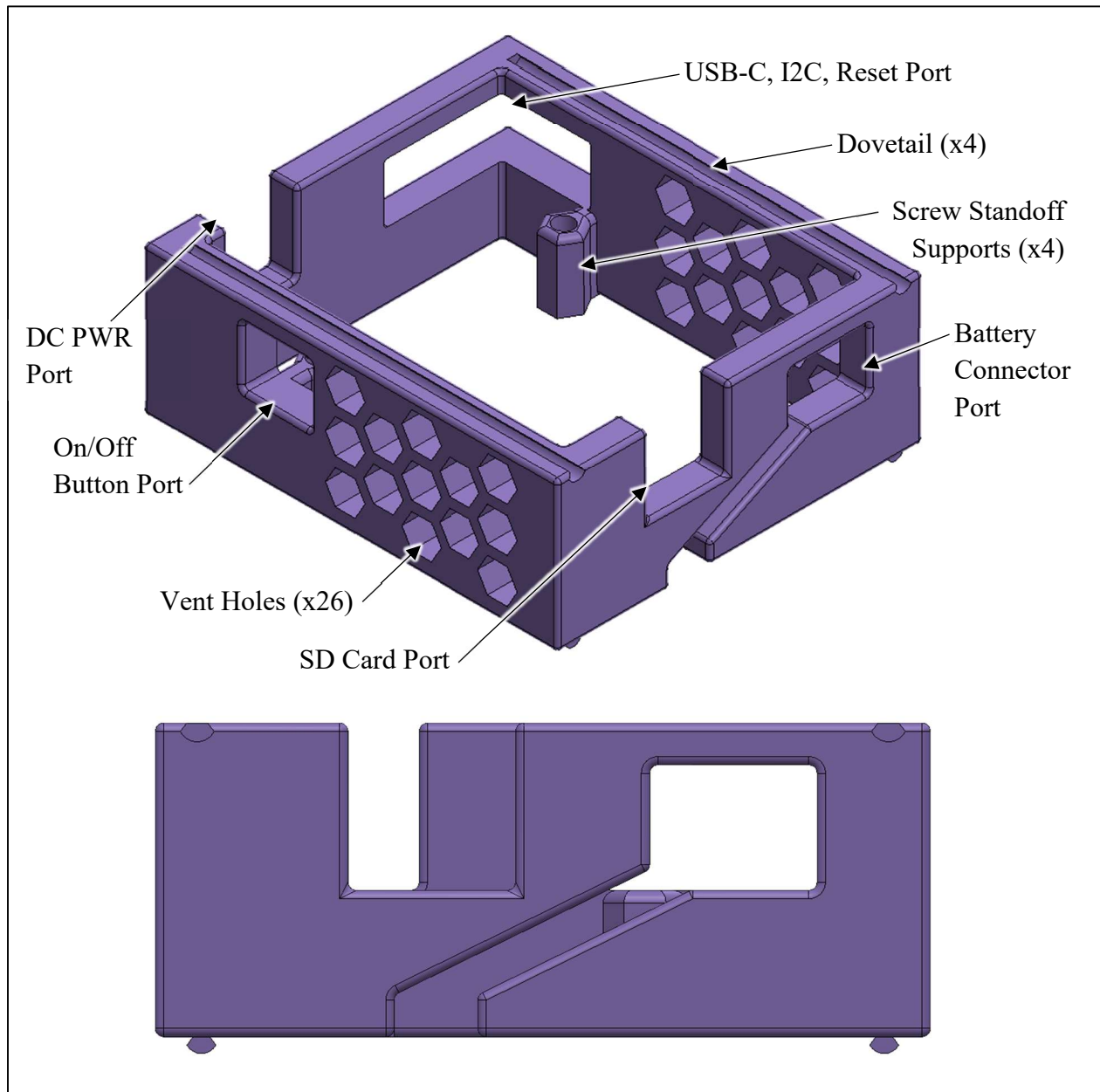


Figure 14: SDACS MCU Housing, Revision C

The MCU Housing uses screw standoff supports to mount the MCU inside the enclosure. There are various ports to allow external access by a connector, tool, or a finger (for buttons/switches). Additionally, the MCU has wire spacing for the battery connector, to allow easy plug-and-unplug of the battery as needed. The MCU Housing has two dovetail pairs: one to connect to the Battery Port layer, and the other to connect to the Lid layer.

3.4.3. Lid

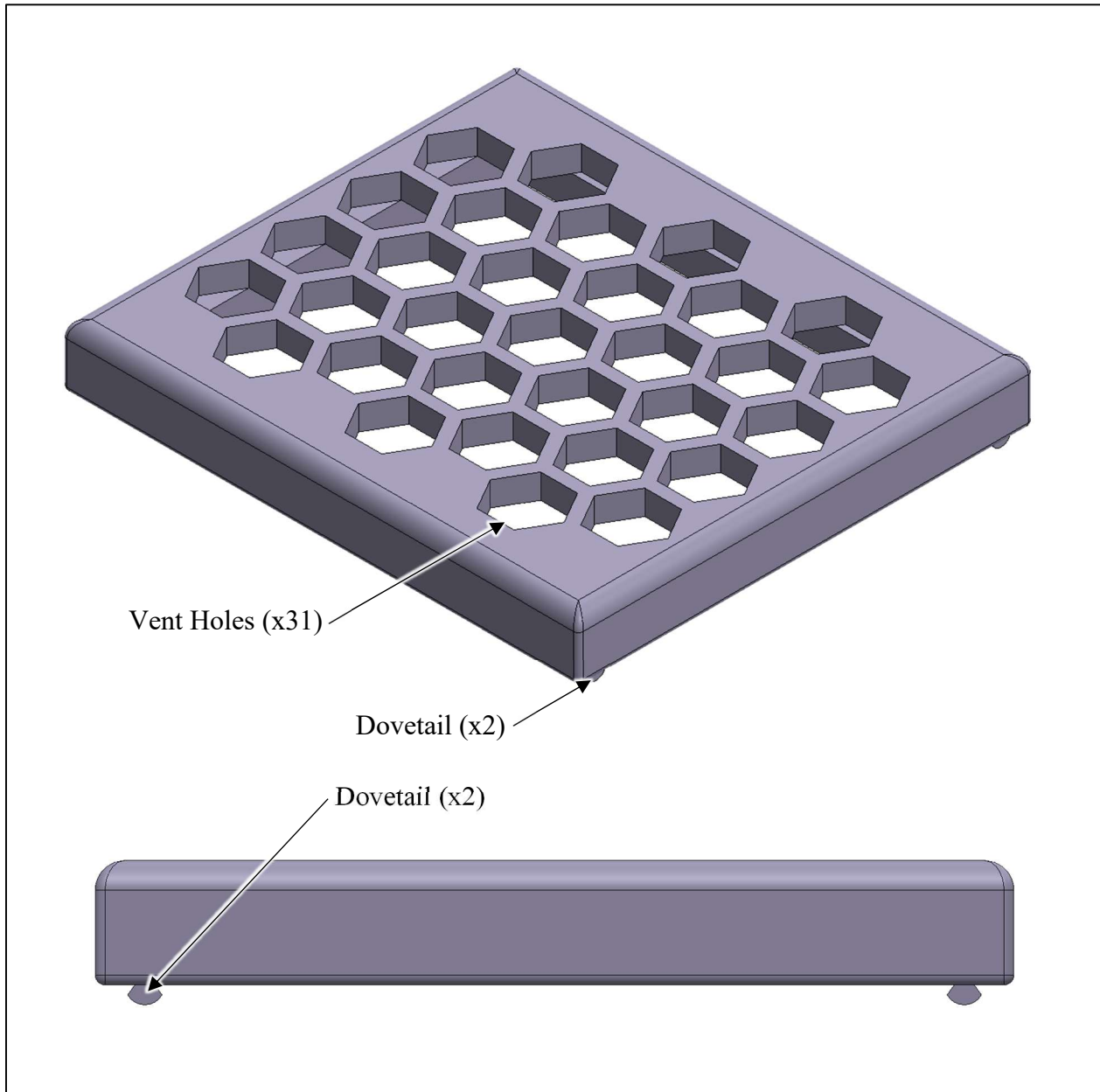


Figure 15: SDACS Lid, Revision C

The Lid provides housing for the custom PCB with vents allowing the microphone and temperature/humidity sensors to access the external environment. The Lid has two dovetails to connect to the MCU Housing Layer.

4. RISK ANALYSIS AND RISK MANAGEMENT PLAN

This section identifies and evaluates the primary risks that could affect the successful completion of the project. Risks are grouped into technical, quality and performance risks, project management risks, organizational risks, and external risks, along with proposed mitigation strategies.

4.1. Technical, Quality, and Performance Risks

4.1.1. Competitive Landscape

There is an established market for professional acoustic measurement tools and audio analysis software. While these products are often costly and complex, their existence introduces the risk that the proposed system may be perceived as insufficiently novel or impactful.

4.1.2. Node Synchronization

Accurate synchronization between independent sensing nodes may be challenging due to network latency and clock drift, potentially affecting comparative measurements during acoustic test events.

4.1.3. Custom PCB Complexity

Designing and fabricating a custom PCB introduces risks related to schedule delays, integration errors, and unnecessary system-on-chip (SoC) over-complication.

4.2. Project Management Risks

4.2.1. Scope Creep

The project involves multiple subsystems (embedded firmware, power electronics, networking, and a web interface), creating a risk of overambition and feature creep.

4.2.2. Time Constraints

The Capstone timeline limits the amount of iteration possible, particularly for hardware and firmware integration.

4.3. Organizational Risks

4.3.1. App and Website Development

The web-based dashboard and application layer introduce uncertainty related to implementation complexity and integration with backend services.

4.4. External Risks

4.4.1. Budget Constraints

The project has a defined budget cap of \$1200, creating a risk of cost overruns due to replacement parts, additional hardware iterations, or unforeseen expenses.

4.4.2. Battery Handling and Safety

The use of lithium-ion / lithium-polymer batteries introduces safety considerations related to charging, handling, and enclosure design.

4.5. Risk Analysis Report

Table 4: Risk Assessment Matrix

Risk	Likelihood (Low-Mod-High)	Impact (Low-Mod-High)
Competitive Landscape	Low	Low
Node Synchronization	Mod	High
Custom PCB Complexity	Low	Low
Scope Creep	Mod	Mod
Time Constraints	Mod	High
App and Website Development	Low	Mod
Budget Constraints	Low	Mod
Battery Handling and Safety	Low	Low

4.5.1. Overall Risk

The overall risk of this project is low with proper accommodation made for seamless hardware-software integration. By testing the hardware, firmware, and software before designing the PCBs, most if not all the technical risks are mitigated. Furthermore, the custom hardware portion has been simplified to ensure a polished prototype system as a proof of concept, with considerations towards a simplified website application. Purchasing extra components for testing purposes has also mitigate a lot of the existing risks.

4.5.2. Risk Priority List

1. Node Synchronization
2. App and Website Development
3. Budget Constraints
4. Time Constraints
5. Scope Creep
6. Competitive Landscape

APPENDICES

See attached documents which support this section which are included in the project drop box.

REFERENCES

- [1] Ableton, “Music production with Live and Push | Ableton,” *Ableton.com*, 2018. <https://www.ableton.com/> (accessed Jan. 26, 2026).
- [2] Apple, “Logic Pro X,” *Apple*, 2019. <https://www.apple.com/logic-pro/> (accessed Jan. 26, 2026).
- [3] Audacity, “Audacity | free, open source, cross-platform audio software for multi-track recording and editing.,” *Audacityteam.org*, May 13, 2019. <https://www.audacityteam.org/> (accessed Jan. 26, 2026).
- [4] “Why Acoustics Matter The Science Behind a Studio Recording Studio,” *Finchley.co.uk*, Nov. 24, 2024. <https://www.finchley.co.uk/finchley-learning/why-acoustics-matter-the-science-behind-a-studio-recording-studio> (accessed Jan. 26, 2026).
- [5] J. Mulcahy, “REW - Room EQ Wizard Room Acoustics Software,” *Roomeqwizard.com*, 2018. <https://www.roomeqwizard.com/> (accessed Jan. 26, 2026).
- [6] “ARC Studio,” *Ikmultimedia.com*, 2026. <https://www.ikmultimedia.com/products/arcstudio/index.php?p=versions#view> (accessed Jan. 26, 2026).
- [7] “SoundID™ - Audio Personalization App,” *Sonarworks*, 2025. <https://www.sonarworks.com/soundid> (accessed Jan. 26, 2026).
- [8] M. R. Bai, Y.-H. Chiang, and P.-C. Wu, “Analysis and synthesis of three-dimensional sound fields using a hybrid microphone array and a 3-layered loudspeaker array,” *Journal of Sound and Vibration*, vol. 443, pp. 666–682, Mar. 2019, doi: <https://doi.org/10.1016/j.jsv.2018.12.018>.
- [9] L. Carneiro and A. Berry, “Three-dimensional sound source diagnostic using a spherical microphone array from multiple capture positions,” *Mechanical Systems and Signal Processing*, vol. 199, p. 110455, Jun. 2023, doi: <https://doi.org/10.1016/j.ymssp.2023.110455>.
- [10] Bluetooth SIG, *Enhancing Bluetooth Location Services with Direction Finding*, Jan. 2019. Accessed: Jan. 27, 2026. [Online]. Available: https://www.bluetooth.com/wp-content/uploads/2019/03/1901_Enhancing-Bluetooth-Location-Service_FINAL.pdf

- [11] *NUMERICAL RECIPES IN FORTRAN 77: THE ART OF SCIENTIFIC COMPUTING*. Cambridge University Press, 1992. Accessed: Jan. 27, 2026. [Online]. Available: <https://phys.uri.edu/nigh/NumRec/bookfpdf/fl2-4.pdf>
- [12] miniDSP Inc., “UMIK-1 USB Measurement Microphone,” miniDSP. Accessed: Jan. 19, 2026. [Online]. Available: <https://www.minidsp.com/products/acoustic-measurement/umik-1>
- [13] Sonarworks Ltd., “SoundID Reference – Pricing and Product Comparison,” Sonarworks. Accessed: Jan. 19, 2026. [Online]. Available: <https://www.sonarworks.com/soundid-reference/pricing>
- [14] Flutter Dev, “Learn Flutter,” Flutter. Accessed: Jan. 19, 2026. [Online]. Available: <https://flutter.dev/learn>
- [15] TDK InvenSense, *ICS-43434 High Performance Digital Microphone Datasheet*, San Jose, CA, USA, 2016.
- [16] F. Alton Everest and K. C. Pohlmann, *Master Handbook of Acoustics*, 6th ed. New York, NY, USA: McGraw-Hill, 2015.
- [17] Analog Devices, *MAX17048/MAX17049 ModelGauge Fuel Gauge IC Datasheet*, Wilmington, MA, USA, 2022.
- [18] Espressif Systems, *ESP32-S3 Series Datasheet*, Shanghai, China, 2023.
- [19] Raspberry Pi Ltd., *Raspberry Pi 5 Product Brief*, Cambridge, U.K., 2023.
- [20] Rational Acoustics, *Smaart v9 User Guide and Technical Overview*, Rational Acoustics LLC, Upper Saddle River, NJ, USA, 2023. [Online]. Available: <https://www.rationalacoustics.com>. Accessed: Jan. 27, 2026.
- [21] Dart Language, “Dart Programming Language,” Dart. Accessed: Jan. 19, 2026. [Online]. Available: <https://dart.dev/>
- [22] MQTT Org., “MQTT – The Standard for IoT Messaging,” MQTT. Accessed: Jan. 24, 2026. [Online]. Available: <https://mqtt.org/>
- [23] HowStuffWorks, “How Bluetooth Works,” HowStuffWorks – Electronics. Accessed: Jan. 24, 2026. [Online]. Available: <https://electronics.howstuffworks.com/bluetooth.htm>
- [24] espressif, “esp-dsp/modules/fft/fixed/dsps_fft2r_sc16_ansi.c at master · espressif/esp-dsp,” GitHub, 2025. [https://github.com/espressif/esp-](https://github.com/espressif/esp-dsp)

- [dsp/blob/master/modules/fft/fixed/dsps_fft2r_sc16_ansi.c#L117](#) (accessed Feb. 02, 2026).
- [25] “2-D fast Fourier transform - MATLAB fft2,” [www.mathworks.com](#).
<https://www.mathworks.com/help/matlab/ref/fft2.html> (accessed Mar. 31, 2026).
- [26] M. Riani, “FSDA - Flexible Statistics Data Analysis toolbox - File Exchange - MATLAB CentralFile Exchange - MATLAB Central,” [Mathworks.com](#), Feb. 15, 2026.
<https://www.mathworks.com/matlabcentral/fileexchange/72999-fsda-flexible-statistics-data-analysis-toolbox/> (accessed Mar. 31, 2026).
- [27] M. Riani, D. Perrotta, and F. Torti, “FSDA: A MATLAB toolbox for robust analysis and interactive data exploration,” *Chemometrics and Intelligent Laboratory Systems*, vol. 116, pp. 17–32, Jul. 2012, doi: <https://doi.org/10.1016/j.chemolab.2012.03.017>.
- [28] Ajeet Kumar Gandhi, V. Mishra, Bhanu Vashistha, M. Rastogi, and Rajesh Vashistha, “Distribution,” Elsevier eBooks, pp. 131–133, Jan. 2023, doi: <https://doi.org/10.1016/b978-0-323-88423-5.00052-2>.
- [29] A. Wentzell, *Project Idea Document*, Conestoga College, 2026.
- [30] A. Wentzell, *Assessment 1 Work Summary*, Conestoga College, 2026.
- [31] A. Wentzell, *SDACS System Diagram*, Conestoga College, 2026.
- [32] A. Wentzell, *MATLAB Research for SDACS*, Conestoga College, 2026.
- [33] A. Wentzell, *Battery Port Design Rev. B*, Conestoga College, 2026.
- [34] K. Dick, *Capstone 1 Bill of Materials*, Conestoga College, 2026.
- [35] K. Dick, A. Wentzell, and D. Toma, *Capstone Project Description*, Conestoga College, 2026.
- [36] K. Dick, A. Wentzell, and D. Toma, *ESE Capstone Functional Block Diagram*, Conestoga College, 2026.
- [37] K. Dick, *Capstone Idea Document*, Conestoga College, 2026.
- [38] A. Wentzell, *MCU Housing Design Rev. B*, Conestoga College, 2026.
- [39] A. Wentzell, *SDACS Enclosure Rough Concept*, Conestoga College, 2026.
- [40] K. Dick, *SDACS Node Calibration and Validation Report*, Conestoga College, 2026.
- [41] K. Dick, A. Wentzell, and D. Toma, *SDACS Project Proposal*, Conestoga College, 2026.
- [42] A. Wentzell, *SDACS Test Report, March 2026*, Conestoga College, 2026.

- [43] K. Dick, A. Wentzell, and D. Toma, *SDACS User Requirements and Project Plan*, Conestoga College, 2026.
- [44] K. Dick, A. Wentzell, and D. Toma, *SMART Centre Git Notes and Process Flow*, Conestoga College, Sep. 4, 2025.
- [45] A. Wentzell, *Top Vent Design Rev. B*, Conestoga College, 2026.